

Trabajo Final de Máster: Aplicación de algoritmos de Machine Learning para predicción de rendimiento de acciones

HUGO GÓMEZ SABUCEDO

hugogomezsabucedo@gmail.com

Máster Big Data, Data Science & Inteligencia Artificial

Curso 2024-2025

Universidad Complutense de Madrid

Índice

1. Introducción al problema	5
1.1. Introducción y objetivos	5
2. Análisis descriptivo y transformaciones	5
3. Modelos: creación y aplicación	10
3.1. Elección de modelos	10
3.1.1. Modelos de árboles	11
XGBoost	11
LightGBM	12
3.1.2. Modelos secuenciales	12
LSTM	12
GRU	13
3.1.3. Modelos convolucionales	13
Conv1D	13
3.2. Preparación y configuración de modelos	13
3.2.1. Creación de datos supervisados	13
3.2.2. <i>Tuning</i> de hiperparámetros y métricas	14
3.2.3. Validación cruzada	15
3.2.4. Entrenamiento de los modelos	16
3.3. Resultados preliminares de los modelos	16
3.4. Bagging: aplicación y resultados	18
4. Resultados	21
5. Conclusiones	24
6. Bibliografía	24
A. Código EDA	25

B. Código para H=4	29
C. Código para H=1	38
D. Código combinado con H=1 y H=4	47

Índice de figuras

1. EDA preliminar	6
2. Valores missing	6
3. Empresas con NaN $\geq 40\%$	7
4. Análisis descriptivo de las 10 primeras empresas	8
5. Empresas según su fuerza estacional	8
6. Descomposición estacional de PPL	9
7. Descomposición estacional de SHEL	9
8. Descomposición estacional de MCK	9
9. Comparación de las series	9
10. Descomposiciones estacionales	9
11. Curvas train-val loss	20
12. Predicciones para horizonte $H=1$	21
13. Predicciones para horizonte $H=4$	23
14. Comparación de predicciones para horizontes $H=1$ y $H=4$	23

1. Introducción al problema

1.1. Introducción y objetivos

El análisis de datos e información financiera es una tarea altamente compleja ya que, al contrario de lo que ocurre en otras series temporales (por ejemplo, de datos de turismo o de viajes), no tienden a ser tan regulares, oscilando mucho ya no sólo de un día a otro, sino en una misma jornada. Además, a la hora de invertir, se busca obtener no sólo el máximo rendimiento en términos absolutos, sino de forma proporcional: es decir, ganar lo máximo posible, invirtiendo no mucho dinero. Es importante también tener en cuenta el posible riesgo de la inversión, así como el entorno que rodea a la misma: si invertimos en una empresa que viene creciendo a un ritmo vertiginoso, es posible que rápidamente deje de crecer o se estanque, con lo que el rendimiento obtenido no sería tan alto como el esperado. Por otra parte, invertir en una empresa que viene reduciendo su cotización no siempre puede ser una buena idea: si esta reducción se produce en un contexto global, podría ser una buena inversión, ya que lo más probable es que vuelva a crecer y se gane dinero; pero si esto se produce en un contexto local, lo más probable es que esa empresa no se recupere, o no lo haga tan rápidamente.

Es por ello que, para poder realizar estas predicciones, se propone este Trabajo de Final de Master, en el que se analizará un dataset disponible en Kaggle que contiene información financiera de las 500 empresas del índice bursátil SP&500 de Estados Unidos. Por tanto, el objetivo de este trabajo será, empleando estos datos, crear y comparar distintos modelos y técnicas de Machine Learning, encontrando el mejor de ellos para predecir el rendimiento de las acciones y poder decidir en qué empresa se debería invertir para obtener un mayor rendimiento.

Para ello, en primer lugar, se realizará un análisis descriptivo de los datos, y se transformarán y limpiarán para asegurar que tienen el formato correcto. Una vez tenemos el dataset adecuado, se crearán los distintos modelos (en este caso, se ha optado por combinar un XGBoost, LightGBM, LSTM, GRU y Conv1D) para entrenarlos sobre una submuestra de los datos, y se compararán las métricas de los mismos, para definir cuáles son los mejores. Sobre estos tres mejores, se aplicará un bagging, con el objetivo de reducir la varianza de los modelos y obtener uno más robusto. Una vez obtenido el modelo final, se reentrenará con los datos de todas las empresas, para poder finalmente realizar las predicciones.

2. Análisis descriptivo y transformaciones

El primer paso es, naturalmente, cargar los datos y realizar un análisis preliminar de los mismos. El código completo de este análisis se encuentra en el anexo A. En este análisis preliminar, en la figura 1 podemos ver que la serie contiene datos para un total de 491 empresas desde el 29 de noviembre de 2018 hasta el 29 de noviembre de 2023; es decir, un total de 5 años exactos. Si analizamos las empresas en función del número de datos que tienen vemos que en el top 10 de empresas con más registros todas tienen el mismo número, 1258, una cifra cercana a lo esperado (teniendo en cuenta que, si en estos cinco años de datos excluimos los fines de semana, días en las que los mercados financieros no abren, tendríamos un total

de 1300 días, a los que habría que restar posibles festivos, etc.). Sin embargo, en el top 10 de empresas con menos datos, hay numerosas empresas con la mitad o más de sus datos faltantes. Posteriormente se explicará como se tratarán estos casos.

```
Empresas totales: 491
Rango de fechas: 2018-11-29 a 2023-11-29

Top 10 empresas con más registros:
Company
A      1258
NTR    1258
ORAN   1258
ON     1258
OKE    1258
ODFL   1258
O       1258
NXPI   1258
NWG    1258
NWS    1258
Name: Date, dtype: int64

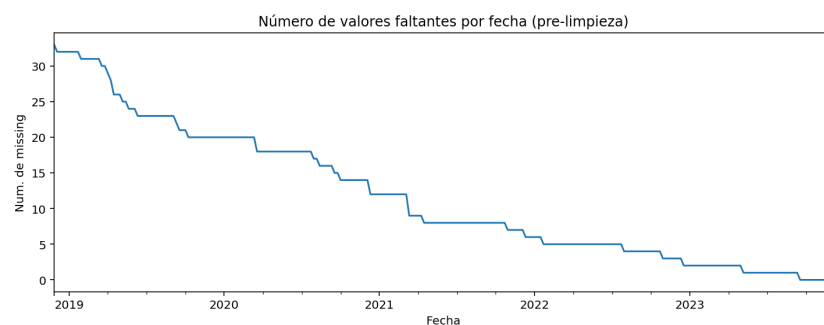
Bottom 10 empresas con menos registros:
Company
CPWG    686
COIN    663
GFS     525
NU       496
CEG     469
HLN     341
MBLY    275
GEHC    240
KVUE    145
ARM       54
Name: Date, dtype: int64
```

Figura 1: EDA preliminar

Visto que el dataframe carece de datos para los fines de semana, lo cual dejaría puntos en la serie temporal sin datos, se ha decidido hacer un resampleo de los mismos con una frecuencia semanal, calculando la media de los datos de esa semana. De esta forma, nos aseguramos de tener una serie de datos consistentes con la que trabajar. Este resampleo, sin embargo, presenta una desventaja, ya que perderemos cierta granularidad en los datos y la posibilidad de registrar mejor ciertas subidas o bajadas bruscas que se produzcan de un día para otro. Pero, por otra parte, como se ha comentado, se gana en consistencia en los datos y también en eficiencia computacional, ya que tenemos que tener en cuenta que el conjunto inicial de datos con el que trabajamos tiene alrededor de 603,000 filas. Además, se ha pivotado el dataframe, de forma que trabajemos con un conjunto de datos cuyo índice sea la fecha (en este caso, semana); las columnas, las distintas empresas; y vayamos almacenando los valores.

```
Company
ARM      0.954198
KVUE     0.881679
GEHC     0.805344
MBLY     0.778626
HLN      0.729008
CEG      0.625954
NU       0.603053
GFS      0.580153
COIN     0.473282
RBLX     0.454198
dtype: float64
```

(a) Valores missing por empresa



(b) Valores missing por fecha

Figura 2: Valores missing

Con estos pasos, se ha reducido el dataset a uno de 262 filas por 491 columnas. El siguiente

ARM	KVUE	GEHC	MBLY	HLN	CEG	NU	GFS	COIN	RBLX	CPNG	SYM	ABNB	DASH
0.954198	0.881679	0.805344	0.778626	0.729008	0.625954	0.603053	0.580153	0.473282	0.454198	0.454198	0.454198	0.40458	0.40458

Figura 3: Empresas con NaN $\geq 40\%$

paso será analizar el porcentaje de valores missing por empresa y por fecha, como se ve en las figuras 2a y 2b. En este caso, observamos que el top 10 de empresas con más valores faltantes tiene un porcentaje de más del 40 %, y vemos que, según nos acercamos al presente, el número de missing por fecha va reduciéndose. Esto nos indica que son empresas que se crearon posteriormente a la fecha de inicio de los datos, o que empezaron a cotizar en bolsa después de esta fecha. Por tanto, para tratar los valores missing, se usarán dos estrategias.

- En primer lugar, se ha definido un umbral máximo sobre el porcentaje de valores missing que puede tener una empresa, que en este caso se ha establecido como un 40 %, y se eliminarán por completo estas empresas. Esto se debe a que se considera que será complejo interpolar los datos para rellenar con los valores faltantes y que, además, no tendría mucho sentido. Como se ha visto, son empresas que han empezado a cotizar en bolsa en años intermedios del dataset, por lo que, si para más de la mitad de los años que analizamos no tenemos sus datos, no tiene mucho sentido incluirlas al análisis. Las empresas que se eliminarán se pueden ver en la figura 3.
- La otra estrategia, para las empresas que presenten menos de un 40 % de valores faltantes, se realizará una interpolación de los mismos. En este caso, indicando el índice como un eje temporal, se realiza una interpolación con `bfill` o `backward fill`. Es decir, se rellenan los valores missing con el siguiente valor válido hacia atrás, de forma que se usa el primer valor válido que haya después de los NaN para rellenarlos con ese valor.

Se han elegido estas estrategias, especialmente la de la interpolación temporal con `bfill`, ya que se han considerado las más adecuadas. Evidentemente, una interpolación con un valor fijo no tendría sentido, ya que distorsiona la serie, al igual que rellenar con medidas estadísticas como la mediana o la moda. Una interpolación polinómica podría no ser tampoco la más adecuada, ya que en series con mucha variabilidad o datos atípicos no funciona adecuadamente. Es por ello que se ha elegido la interpolación temporal, ya que mantiene la coherencia temporal de los datos, algo crucial. De esta forma, tras la aplicación de estas técnicas, tenemos unos datos perfectamente limpios.

El siguiente paso es realizar un muy breve análisis descriptivo de los datos, en este caso para las 10 primeras empresas. Se puede observar una variabilidad en los precios, con empresas como AAPL (Apple) con desviaciones estándar elevadas teniendo en cuenta las medias. En general, las medias y medianas (50 %) son cercanas, lo que indica que, pese a las fluctuaciones, estas no son excesivamente elevadas. De todas formas, estos datos nos podrían indicar una ligera volatilidad, lo cual se debe tener en cuenta en los análisis futuros,

A continuación, se analiza la fuerza estacional de las series, para ver qué empresas presentan una mayor fuerza estacional. Para ello, se hace la descomposición estacional de la serie, teniendo en cuenta que trabajamos con series temporales anuales (un periodo de 52 semanas), y calculamos la fuerza estacional, que es la relación entre la varianza del componente estacional

Company	count	mean	std	min	25%	50%	75%	max
A	262	112.046	30.0516	61.0184	80.0922	118.561	134.803	176.669
AAPL	262	118.915	47.2635	35.5754	70.5751	130.659	155.707	195.309
ABBV	262	104.94	33.2663	53.1197	74.1516	100.88	138.726	163.54
ABEV	262	2.94515	0.619154	1.76905	2.54337	2.77898	3.34223	4.51929
ABT	262	98.2452	17.1848	61.8576	81.7266	102.317	110.96	135.703
ACGL	262	45.4913	15.6241	22.98	34.155	41.62	47.52	86.6
ACN	262	249.765	62.9826	127.917	190.243	263.051	301.962	402.575
ADBE	262	417.346	116.001	208.8	319.16	407.605	500.067	688.37
ADI	262	139.015	32.5626	75.217	107.107	147.315	164.421	197.481
ADM	262	58.0849	19.3557	27.946	38.7064	57.0652	75.3234	95.1963

Figura 4: Análisis descriptivo de las 10 primeras empresas

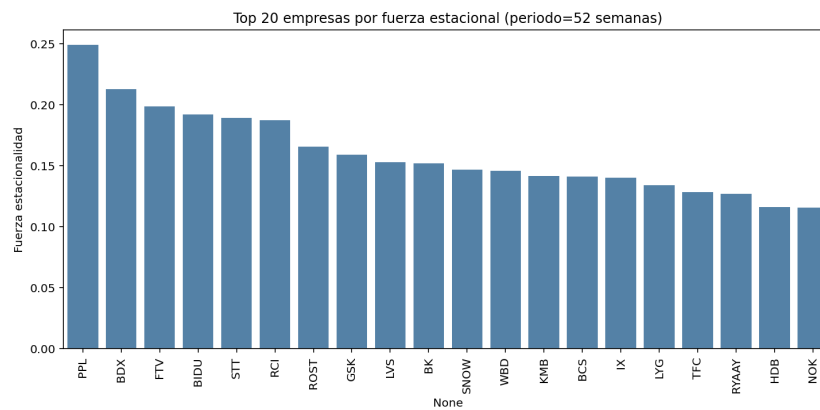


Figura 5: Empresas según su fuerza estacional

de la serie y la varianza total de la serie. Así, se obtiene el gráfico de la figura 5, donde se ven las 20 empresas con mayor fuerza estacional. Esta clasificación la emplearemos para calcular la descomposición de la serie en sus componentes, como se ve en la figura 10.

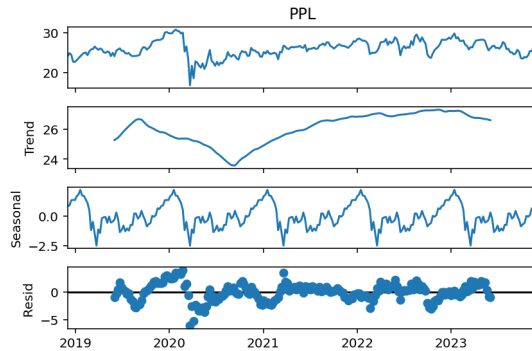


Figura 6: Descomposición estacional de PPL

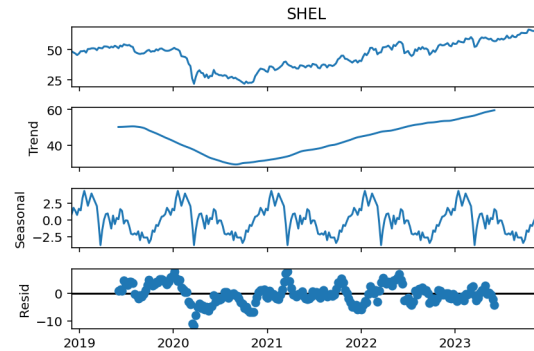


Figura 7: Descomposición estacional de SHEL

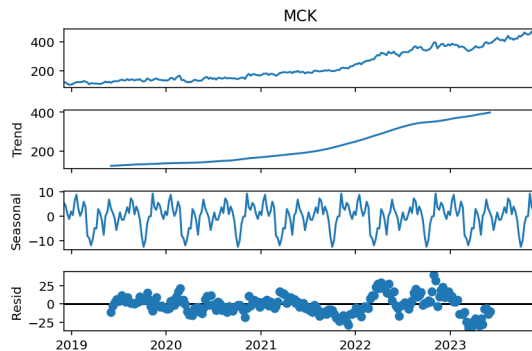


Figura 8: Descomposición estacional de MCK

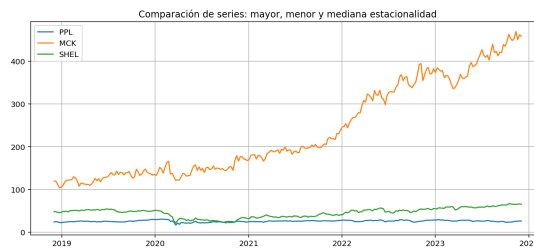


Figura 9: Comparación de las series

Figura 10: Descomposiciones estacionales

Se ha elegido la serie con mayor fuerza estacional (PPL, figura 6), con menor fuerza estacional (MCK, figura 8) y la mediana (SHEL, figura 7). Además, en la figura 10, podemos ver las tres series dibujadas juntas (aunque cada serie individual se puede ver también en su figura respectiva). La descomposición temporal de las series nos da cuatro gráficos distintos:

1. El **componente observado**, que es la serie temporal tal y como es, con todos sus valores.
2. El **componente de tendencia**, que captura la dirección a largo plazo de la serie. En este caso, se ve una tendencia creciente en las tres empresas, pero con ligeras diferencias. Mientras que la primera empresa se ve una tendencia creciente, que se vio interrumpida por la pandemia, para luego sufrir de nuevo un crecimiento explosivo y un estancamiento; en la segunda empresa se observa simplemente la caída debida a la pandemia y el posterior crecimiento, en una tendencia ascendiente que aún no se ha cortado; y en la última empresa, se ve un crecimiento constante, menor o menos abrupto los primeros años (coincidentes también con la pandemia)

3. El **componente estacional**, que captura los patrones o ciclos en los datos. Estos patrones pueden deberse a numerosas causas, como el clima, festividades, las estaciones, etc. Representa las fluctuaciones que ocurren, para el mismo período (en este caso, anual), en cada ciclo (por ejemplo, se observa que, por lo general, al inicio del año, los precios sufren una bajada, precedida de una subida en los últimos meses del año). Es crucial para el análisis, ya que nos permite saber si podremos predecir estos patrones correctamente.
4. El **componente residual**, la variabilidad aleatoria que no se explica ni por la tendencia ni por la estacionalidad. Se corresponde con anomalías o eventos únicos, ruido, por lo cual, como es esperable, nuestras series presentan ciertos residuos. Sin embargo, lo importante es que estos estén incorrelados, que sean aleatorios. Si aplicamos la prueba de Ljung-Box sobre los residuos, que evalúa precisamente si existe una autocorrelación significativa, obtenemos p-valores extremadamente pequeños, del orden de 10^{-100} . En este caso, nos indicaría que los residuos no son aleatorios. Sin embargo, esto puede deberse a que esta descomposición se hace usando modelos relativamente simples, lineales, y que podríamos necesitar modelos más complejos para analizar series complejas como las que tenemos.

Por último, una vez que hemos realizado el análisis al completo y que hemos limpiado los datos, vamos a realizar una exportación del dataset ya limpio, para poder usarlo como base de trabajo en los sucesivos códigos y no tener que realizar la limpieza cada vez que se ejecutan.

3. Modelos: creación y aplicación

En esta sección se comentará el bloque principal de este trabajo: los modelos. Se explicará cuáles se han elegido y el porqué, comentando sus ventajas y desventajas. Se comentará también como se han creado y entrenado, así como los parámetros empleados, explicando las decisiones tomadas. Se analizarán los resultados y métricas obtenidas, y se explicará el proceso de bagging empleado para mejorar los modelos, con sus correspondientes métricas y resultados.

3.1. Elección de modelos

Para este trabajo se han elegido cinco modelos principales, cada uno de los cuales aporta numerosas ventajas y desventajas. Además, se han desarrollado tres scripts distintos como parte de la elección no sólo del mejor modelo, sino de la forma de predecir los datos. Por una parte, tenemos en el anexo B un código que entrena los modelos y realiza predicciones para una predicción a corto plazo, de una semana; por otra parte, en el anexo C, un código que realiza el entrenamiento y predicción en un horizonte a medio plazo de 4 semanas; y por último, en el anexo D, un framework combinado.

El motivo de haber desarrollado estos tres códigos, aunque realmente son similares entre ellos, es un enfoque progresivo del problema. Aunque en todos los casos se emplean los mismos modelos, la diferencia radica en el horizonte temporal de salida: mientras que en el caso de $H=1$ los modelos se centran en predecir un único paso, los modelos con $H=4$ deben predecir a un mes vista, cambiando el enfoque del problema a una predicción multihorizonte. Esto, pese a

mantener la estructura de los modelos, incrementa la dificultad del aprendizaje de los mismos y suele afectar también a la precisión de las predicciones, ya que tienen el reto no sólo de predecir con mayor o menor exactitud el siguiente valor de la serie, sino la tendencia y evolución de la serie. Finalmente, el framework combinado se concibió como una manera de tener una mejor visión de los resultados de los modelos, ya que evidencia cuál de los dos casos genera mejores predicciones. De este modo, la combinación de las tres versiones enriquece también el análisis.

En lo que a los modelos respeta, se han elegido cinco tipos diferentes: XGBoost, LightGBM, LSTM, GRU y Conv1D.

3.1.1. Modelos de árboles

Los modelos de árboles se basan en la combinación, mediante *gradient boosting*, de árboles de decisión “débiles” secuencialmente, de forma que cada árbol nuevo corrige los errores del anterior. Aprenden minimizando una función de pérdida y cada iteración añade un árbol entrenado sobre los residuos. Son buenos para capturar no linealidades e interacciones entre variables, pero no comprenden el orden temporal, por lo que les tendremos que proporcionar ventanas como features, como se explicará en 3.2. Dentro de este tipo se encuentran los modelos de XGBoost y LightGBM.

En el contexto de este trabajo, son adecuados ya que, tras la creación de las ventanas temporales, los datos presentan una estructura tabular, con la que cada fila representa un bloque de W semanas y la variable objetivo se corresponde con la semana siguiente (o las cuatro siguientes). Esto es fácilmente manejable por estos algoritmos, capturando las relaciones no lineales entre ellos.

XGBoost

XGBoost, o eXtreme Gradient Boosting, es un algoritmo de aprendizaje supervisado que implementa una técnica llamada *gradient boosting*. En esta técnica se entrenan secuencialmente múltiples árboles de decisión, de forma que cada nuevo árbol corrija los errores cometidos por los anteriores optimizando una función de pérdida. Su principal ventaja es que usa la memoria muy eficientemente y tiene una estructura optimizada para que las predicciones y las construcciones de los árboles sean fácilmente escalables. Esta técnica, además permite controlar el sobreajuste si se controlan o modifican adecuadamente los hiperparámetros, pero presenta la desventaja de necesitar ventanas muy largas para ver bien el pasado y puede ser muy costoso con árboles muy profundos.

Los parámetros que se pueden configurar para estos modelos, y los que se usarán en este trabajo, son tres:

1. **n_estimators**: representa el número de árboles; a mayor número de árboles, mejores resultados, pero también hay más riesgo de *overfitting*.
2. **max_depth**: representa la profundidad máxima de cada árbol, y permite limitar la complejidad.

3. **learning_rate**: representa cuánto aporta cada nuevo árbol; cuanto más pequeño es el valor, más robusto es el modelo, pero más lento de entrenar.

LightGBM

LightGBM, o Light Gradient Boosting Machine, es otra forma de implementación de boosting de árboles, diseñado con el objetivo de ser eficiente tanto en el tiempo que requieren para ser entrenados como en el uso de la memoria. Usa histogramas y crecimiento *leaf-wise* que, al contrario de los modelos tradicionales (o *depth-wise*, que trabajan dividiendo horizontalmente los árboles en cada nivel, generando árboles más anchos), trabaja seleccionando la hoja más prometedora (con mayor *gain*) para dividir el árbol. Esto acelera mucho la creación del árbol, pero también implica mayor riesgo de sobreajuste.

Para este modelo, se configurarán también los mismos parámetros que en anterior.

3.1.2. Modelos secuenciales

Estos modelos procesan los datos en un orden temporal, manteniendo un estado oculto o memoria. Aprenden combinando, en cada paso, la entrada actual con el estado anterior, produciendo un nuevo estado, con compuertas controlando qué información se mantiene o se olvida. Son muy buenas para capturar dependencias en series temporales, tanto a corto como a largo plazo, así como estacionalidades de las mismas. Sin embargo, son más costosas de entrenar y sensibles a escalados de los datos y a *overfitting*. Son los modelos más complejos. Dentro de este tipo se encuentran los modelos de LSTM y GRU.

En el contexto de este trabajo, con series con información de numerosos meses y en las que cambios repentinos en los datos son habituales, son modelos muy apropiados, ya que permiten tener en cuenta no sólo los datos de las semanas más inmediatamente recientes, sino también la trayectoria de la serie y su inercia. Esto presenta una ventaja importante sobre los modelos de árboles, permitiendo ajustar los datos con la información que tienen del largo plazo y saber si venían en una tendencia creciente o decreciente.

LSTM

Las LSTM, o Long Short-Term Memory, son un tipo de red neuronal recurrente (RNN) que incorporan una memoria a largo plazo con el objetivo de recordar información durante secuencias largas. Están compuestas por células, cada una de las cuales tiene dos estados: un estado oculto, que es el que se transmite de paso en paso y se usa para generar la salida; y un estado de celda (o cell state), que es el que actúa como memoria a largo plazo. El flujo de la información se controla mediante tres *gates*: la *forget gate*, que decide qué parte de la memoria anterior se descarta; la *input gate*, que decide qué nueva información se guarda; y la *output gate*, que controla qué parte de la memoria influye en la salida.

Los parámetros que se ajustarán para este modelo son:

1. **units**: el número de células ocultas de la red.

2. **dropout**: una forma de regularización, que inactiva ciertas neuronas para que las restantes aprendan de forma más robusta.

GRU

Las GRU, o Gated Recurrent Unit, son una variante simplificada de las LSTM, que mantienen tanto la idea de la memoria como de las puertas, pero fusionando algunas de ellas, con lo que logran reducir los parámetros y acelerar el entrenamiento. Cadaa unidad GRU tiene ahora un único estado oculto, controlado por dos puertas: la *update gate*, que decide cuánto se mantiene del estado anterior; y la *reset gate*, que decide cuánto se olvida.

En este caso, también se ajustarán los mismos parámetros que para las LSTM.

3.1.3. Modelos convolucionales

Estos modelos presentan una complejidad intermedia entre los modelos de árboles y los secuenciales. Su idea básica es aplicar filtros o kernels que van barriendo y analizando la secuencia y detectando patrones locales, bien sean tendencias u oscilaciones, en las ventanas. Aprenden usando pequeñas cuadrículas, llamadas *kernels* o núcleos, que se van desplazando sobre los datos y generando mapas de activación. Su ventaja es que permiten combinar varias capas, cada una detectando un patrón o tendencia específica. Tienen la ventaja de capturar muy bien patrones locales repetitivos y de corto plazo, además de ser rápidos y fácilmente paralelizables. Dentro de este tipo de modelos se usará el Conv1D.

Conv1D

Las Conv1D, o Convolutional Neural Network 1D, son un tipo de red neuronal que, en lugar de procesar los datos paso a paso, aplican filtros deslizantes que los van recorriendo y detectando patrones locales. Funcionan con filtros convolucionales, que no son más que vectores de pesos que se desplazan sobre la secuencia de estado y van calculando, para cada posición, una combinación lineal de los valores de la ventana. Cada filtro aprende un patrón específico (que puede ser tendencias ascendentes en el último mes, por ejemplo) de forma que, combinando el uso de varios filtros en paralelo, se aprenden numerosos patrones. Tras la convolución, se usan capas adicionales para aplanar y combinar las características.

Para este modelo se ajustarán, al igual que los anteriores, las **units** y el **dropout**, así como el **kernel_size**, que modifica la longitud del kernel en pasos de tiempo.

3.2. Preparación y configuración de modelos

3.2.1. Creación de datos supervisados

Un aspecto crucial a tener en cuenta a la hora de comenzar a trabajar con los modelos es la necesidad de transformar los datos en un problema supervisado, lo que se conoce como *ventanización* o *windowing*. Esto es necesario porque los modelos que se entrenan requieren

recibir los datos en forma de pares de entrada-salida, y la serie de datos en crudo (una simple columna de precios a lo largo del tiempo) no lo es. De esta forma, el dataset restante tendrá la siguiente forma: $X_t = [y_{t-W+1}, y_{t-W+2}, \dots, y_t]$

Para los modelos de árboles, ya que no entienden de orden temporal, si no se proporcionan los datos en forma de ventana sólo verían una columna de valores sin contexto. En el caso de las RNN, aunque pueden procesar secuencias, necesitan un tamaño fijo de entrada, con el que saber cuántos pasos para atrás mirar para hacer la predicción. Para esto, se ha realizado una función que, en base al tamaño de ventana definido y al horizonte de predicción, genera unos nuevos datos donde la entrada son las últimas W observaciones, y la salida es 1 o 4 (en función de H) observaciones futuras.

Respecto a la elección del tamaño de la ventana, se ha optado por un tamaño de 25 semanas (aproximadamente 6 meses), ya que se ha considerado un tamaño equilibrado entre la captura de suficiente contexto histórico para reflejar correctamente las tendencias a largo plazo y la reducción del ruido. Una ventana muy corta, de unas 3-5 semanas, haría que el modelo sólo observase fluctuaciones recientes, y que no detectase correctamente las tendencias. Una ventana muy larga, de por ejemplo 100 semanas, incluiría datos demasiado antiguos, que diluiría la información más reciente (crucial, para detectar tendencias inmediatas) y ralentizaría mucho el entrenamiento. Es, por tanto, un punto intermedio. Además, para el horizonte de predicción que se ha definido (bien sea el de una semana o el de cuatro), es un tamaño adecuado, ya que no se necesita información, por ejemplo, de dos años, para predecir el rendimiento de la próxima semana.

3.2.2. *Tuning* de hiperparámetros y métricas

Como se comentó en 3.1.1, cada uno de los modelos que se emplean tienen una serie de hiperparámetros que condicionan no sólo su capacidad de aprendizaje, sino cómo se comportan frente al sobreajuste de los datos o qué límite tienen a la hora de parar. A diferencia de los parámetros internos que tienen cada uno de ellos, que se van ajustando automáticamente, los hiperparámetros se deben establecer de forma externa.

Para ello, se ha aplicado una *grid search*, en la que se define un espacio discreto de valores para cada uno de los posibles hiperparámetros y se entrena el modelo sucesivamente con cada una de las combinaciones. Este es un enfoque que, si bien es computacionalmente costoso, permite explorar una serie de valores razonables, obtenidos a partir de experimentos realizados tanto sobre los mismos datos como analizando los valores que se suelen usar más frecuentemente. Cabe destacar que los valores que se presentan en los códigos de los anexos no son todos con los que se ha probado, ya que previamente se ha hecho una búsqueda más extensa, con el objetivo de ajustarlos y eliminar aquellos con peores resultados.

Relacionado con los hiperparámetros, conviene hablar de las métricas empleadas para medir el rendimiento de los modelos. En este caso, se ha definido una función propia que computa a la vez distintas métricas para el mismo modelo:

- R^2 (Coeficiente de determinación). Es la métrica por excelencia, que mide la proporción de la varianza de la variable dependiente que es explicada por el modelo. Valores próximos

a 1 indican que el modelo explica perfectamente los datos, mientras que próximos a 0 indican que el modelo no mejora respecto a predecir siempre la media. Es el criterio que usaremos como principal a la hora de determinar el mejor modelo.

- **MSE** (Mean Squared Error). Se calcula como la media de los cuadrados de las diferencias entre los valores reales y los predichos, y es útil ya que penaliza fuertemente los errores grandes (debido a la elevación al cuadrado), aunque no es interpretable en las mismas unidades que los datos originales
- **MAE** (Mean Absolute Error). Similar al anterior, pero mide únicamente la media de las diferencias absolutas. Es más robusto frente a valores atípicos o excesivamente altos, e indica, en promedio, cuánto se desvía la predicción de los valores reales, con la ventaja de hacerlo en las unidades de la variable.
- **RMSE** (Root Mean Squared Error). Es simplemente la raíz cuadrada del MSE, lo que lo hace más interpretable al devolver el error medio, ahora sí, en las mismas unidades que los datos.
- **MAPE** (Mean Absolute Percent Error). Mide el error relativo en porcentaje. Tiene la ventaja de ser intuitivo (por ser un porcentaje), pero no funciona bien con valores próximos a cero. En este caso, nos sirve para comparar empresas con diferentes escalas de precios
- **EVS** (Explained Variance Score). Mide cuánta varianza de los datos es explicada por el modelo, de forma similar al R^2 , pero más centrada en la dispersión de los errores.
- **DA** (Directional Accuracy). Se emplea en algunos casos para evaluar si el modelo predice correctamente la dirección del cambio (es decir, si el modelo acierta cuando la serie sube o baja respecto al valor anterior). Es muy útil, ya que en nuestro caso seguramente sea más útil predecir bien la tendencia que el valor exacto.

3.2.3. Validación cruzada

A la hora de evaluar los modelos, es importante comprobar que las métricas que hemos obtenido reflejen su capacidad de generalización a datos que no se le han proporcionado como conocimiento. En problemas de regresión o clasificación, suele ser suficiente con realizar una validación cruzada de los datos, dividiéndolos en particiones de entrenamiento y de test. Sin embargo, en datos con una estructura temporal, como los nuestros, esta validación cruzada es inapropiada, ya que rompería la propia secuencialidad de los datos, produciendo un *data leakage* (cuando los modelos se entrenan con información futura).

Es por ello que, en este caso, se ha optado por emplear una validación cruzada, pero temporal (denominada `TimeSeriesSplit`). Este método genera divisiones sucesivas de los datos, en donde cada conjunto de entrenamiento contiene únicamente observaciones pasadas, y el conjunto de validación contiene valores futuros, que no se emplean en el entrenamiento. Así, combinamos el respeto por el orden cronológico y se simulan condiciones reales de predicción.

En este caso, se han empleado un número variable de particiones en función del tipo de modelo con el que se trabaja. Para modelos de árboles de decisión, debido a que presentan un menor coste computacional, se han utilizado cinco particiones; mientras que para los modelos

secuenciales y convolucionales, de mayor complejidad, se han empleado únicamente tres particiones. Cada partición se evalúa con las métricas comentadas en 3.2.2, y los resultados obtenidos se corresponden con la media de las métricas a lo largo de todos los *folds*.

3.2.4. Entrenamiento de los modelos

Este proceso es la parte central del trabajo, pues es donde se entrena a los algoritmos para que identifiquen los patrones y realicen las predicciones. Como se comentó en 3.2.1, se emplearán estos datos transformados, tanto para los casos en el que se emplea el horizonte de 1 semana como para el de 4, ya que se ha diseñado una función que crea los datos teniendo en cuenta ambos parámetros. Sin embargo, estos datos no son los que se empleen directamente para entrenar los modelos, sino que primero deben escalarse, empleando un `StandardScaler`. Esto se hace para homogeneizar las escalas de los datos, y evitar que empresas con precios de las acciones muy elevados dominen los modelos y tiren de las predicciones. Además, se realiza un aplanado de los datos en vectores para los modelos de árboles, mientras que se conservan como tensores tridimensionales para las redes neuronales, para ser consistentes con las dimensiones de los datos que necesitan cada uno de los modelos.

Una particularidad a destacar de esta fase de entrenamiento es que el entrenamiento y ajuste inicial de los hiperparámetros se realiza inicialmente sobre un subconjunto aleatorio de un 40 % de las compañías. Esto responde a varios motivos. Por una parte, eficiencia computacional; dado que estamos trabajando con aproximadamente 500 empresas, entrenar cada uno de los modelos (en este caso, y con la criba de hiperparámetros que ya hemos hecho, entrenamos 8 modelos tanto en XGBoost y LightGBM, 6 en LSTM y GRU y 12 en Conv1D) con todos los datos consumiría muchos recursos, tanto a nivel computacional como temporal. Por otra parte, al seleccionar de forma aleatoria las empresas nos aseguramos de que sea una muestra heterogénea (en vez de elegir, por ejemplo, el top de empresas con mayor cotización u otro criterio).

Una vez ajustados todos los modelos sobre este subconjunto, podemos elegir los más prometedores en términos de sus hiperparámetros, para aplicarles el bagging y poder reentrenarlos de nuevo, ahora sí, con todos los datos. Además, no corremos el riesgo de que se produzca un sobreajuste en los datos en el modelo final, ya que esta fase inicial únicamente se usa para elegir el modelo más prometedor, y luego reentrenamos el modelo, ya sí, teniendo en cuenta todos los datos.

3.3. Resultados preliminares de los modelos

En este apartado, analizaremos los resultados preliminares obtenidos entre los cinco modelos, para determinar los mejores, y aquellos a los que se les aplicará el *bagging*. Para ello, compararemos las métricas que se explicaron en 3.2.2, veremos si hay relaciones entre los modelos y aplicaremos *bagging* a los mejores.

Así, en el cuadro 1 podemos ver los resultados de las métricas para las predicciones realizadas con un horizonte de una semana, mientras que en el cuadro 2 podemos ver las métricas para las predicciones realizadas con un horizonte de cada semana, ordenados en

Modelo	MSE	RMSE	MAE	R2	MAPE	EVS
Conv1D	1093.488254	30.505075	11.599173	0.981727	10.36545	0.981755
XGBoost	94997.178863	147.651011	31.917192	0.854591	5.743488	0.859903
LightGBM	95040.103876	148.86174	31.931191	0.852904	4.781479	0.858218
GRU	148093.259157	274.145885	61.88736	0.43245	16.99539	0.468539
LSTM	151292.014625	276.373591	63.479611	0.425963	18.0122	0.455708

Cuadro 1: Resultados de los modelos (H=1)

Modelo	MSE	RMSE	MAE	R2	MAPE	EVS
Conv1D	1482.329199	35.31938	13.317512	0.975987	12.895549	0.976049
XGBoost	95709.451337	151.978414	34.332828	0.847299	7.828946	0.85269
LightGBM	96230.201717	153.48642	34.53518	0.844269	7.768551	0.849684
GRU	148269.588843	273.933054	62.878443	0.429544	14.947863	0.465223
LSTM	148871.35456	275.026711	64.14006	0.422786	19.796763	0.461598

Cuadro 2: Resultados de los modelos (H=4)

ambos casos por el R^2 . Se observa un patrón claro, donde se ve que Conv1D es el mejor en ambos casos, con mucha diferencia sobre el resto; los modelos de árboles (LightGBM y XGBoost) se sitúan un peldaño por debajo, también con buenas métricas; mientras que los peores son los modelos secuenciales, tanto LSTM como GRU, que, en diferente orden, presentan el peor rendimiento.

El modelo Conv1D se muestra como el más preciso en ambos horizontes, tanto en H=1 (con un R^2 de 0.981) como en H=4 (con un R^2 de 0.975), lo cual significa que explica más del 98 % de la varianza de los datos, lo cual refleja un buen ajuste. Además, tiene un error absoluto y relativo bastante bajo, de un 11.5 % y un 10.4 % respectivamente, los más bajos de entre todos los modelos, lo que significa que las predicciones son muy cercanas a los resultados reales. Esto se refuerza con el alto valor de la EVS, del 98 %.

Este era el resultado esperable, especialmente en el contexto de este trabajo, ya que las convoluciones son muy útiles para capturar los patrones locales de los datos de forma muy eficiente en series temporales, sin necesidad de tener dependencias de largo plazo muy largas. Además, con una arquitectura más sencilla y menos propensa al sobreajuste, se facilita la generalización, lo que explica que el rendimiento sea prácticamente idéntico en ambos horizontes.

Sin embargo, hablando de sobreajuste, unas métricas tan perfectas nos podría dar lugar a pensar que el modelo haya caído en un sobreajuste de los datos, sobre todo teniendo en cuenta la comparativa con el resto de modelos. Este tema se analizará más en 3.4, pero hay varios indicios que nos indican que podría no haber sobreajuste. Se ha realizado una validación cruzada temporal, y el rendimiento se mantiene alto tanto en entrenamiento como en test. Además, el comportamiento del modelo es consistente en ambos horizontes temporales, lo que indica una estabilidad del rendimiento, que sería más difícil en un modelo con un horizonte temporal más amplio. Como se ha comentado, los modelos convolucionales son especialmente buenos para capturar patrones cortos, con tendencias locales y cambios abruptos, que precisamente son la naturaleza de las series financieras.

En el otro extremo de rendimiento tenemos a los modelos secuenciales, tanto LSTM como GRU, que ofrecen un rendimiento abrumadoramente inferior en ambos escenarios, con un valor del R^2 que apenas llega al 0.4. Las métricas de error confirman esto también, con un MAE en torno a 60 y un MAPE en torno al 20 %. Aunque estos modelos están pensados para manejar dependencias a largo plazo, su mal desempeño podría ser previsible, ya que los patrones que encontramos en las series financieras suelen ser más a corto plazo, lo que no beneficia a estos modelos. Además, son muy sensibles al *tuning* de los hiperparámetros, que debe ser muy fino, ya que si no disponen de una configuración óptima tenderán o bien a sobreajustar o, como es este caso, a no converger adecuadamente. Es posible que, con una búsqueda más exhaustiva de hiperparámetros, analizando muchas más combinaciones, se llegara a obtener modelos un poco mejores, pero esto no ha sido posible, en parte, debido a la falta de recursos computacionales.

Por último, con un buen rendimiento, aunque un escalón por debajo de los modelos convolucionales, tenemos a los modelos de árboles, LightGBM y XGBoost, que obtienen unos resultados más que aceptables. Con un R^2 que se sitúa en torno al 0.85, además en ambos horizontes de predicción, tienen un muy buen rendimiento. Un hecho destacable es que, pese a tener unas métricas de error mucho más elevadas que el modelo convolucional (un RMSE, en $H=1$, de aproximadamente 150 frente al 30 que tenía Conv1D, y un MAE de 31 frente a 11), el MAPE o error porcentual es inferior, aproximadamente la mitad (para Conv1D teníamos un 10.36 % en $H=1$, mientras que el de XGBoost es de 5.74 %, y el de LightGBM es aún más pequeño, de 4.78 %). Esto nos indica que, aunque los errores son mayores en términos absolutos, en término relativo se equivoca menos. Teniendo en cuenta que los datos son el valor en dólares de las cotizaciones de las empresas, quiere decir que se equivocan más en empresas con valores más altos, donde el impacto relativo es menor (es decir, no es lo mismo equivocarse por 10 dólares en una acción que cotiza a 1000, que sería un 1 % de error; que hacerlo en 1 dólar en una acción que cotiza a 5, que sería un 20 %).

Este comportamiento también era esperable, ya que estos algoritmos captan muy bien relaciones no lineales entre los datos y patrones complejos, que son los que se producen en este caso. Sin embargo, frente a horizontes más largos, pueden perder precisión, como se ha observado, ya que al no modelar de forma nativa la secuencia temporal hace que pierdan parte de la información estructural de la serie.

Una vez analizados todos los resultados, se elegirán para aplicar el bagging el modelo de Conv1D, así como el de XGB, por ser los que mejores métricas presentan.

3.4. Bagging: aplicación y resultados

Con los dos mejores modelos seleccionados, se aplicarán técnicas de ensamblado de modelos con el objetivo de combinar los modelos de forma que se mejoren las predicciones realizadas por los mismos. Aunque en este caso tenemos un modelo, el Conv1D, que ya tiene unas métricas buenas, nos servirá también para ver si el modelo está sobreajustando o no. El *bagging*, también conocido como *bootstrap aggregation*, es una técnica consistente en, una vez entrenados los modelos de forma independiente, promediar sus predicciones, para obtener un modelo combinado que reduzca la varianza de las predicciones, mejorando su capacidad de generalización. Al combinar resultados de diversos modelos, se atenúan los errores propios de cada uno de ellos, generando predicciones estables y con mejor generalización, ya que se

disminuye el riesgo de sobreajuste. Además, mejora la robustez, ya que se evita depender excesivamente de los sesgos de un modelo concreto. Su principal ventaja es su sencillez de implementación, y la notable mejora que proporciona en los resultados de los modelos. A cambio, conlleva un mayor coste computacional y, además, puede ser innecesario en modelos que ya obtienen buenos resultados, como el nuestro. Por lo tanto, se deberá analizar si mejora los resultados, cuánto, y si esta mejora compensa el coste.

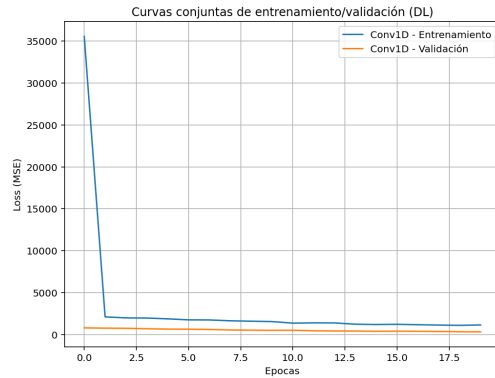
En este estudio, se ha aplicado en dos niveles. Por una parte, el bagging intra-modelo, en el que se ha entrenado cada modelo de forma independiente un total de 5 veces (n_bags). Esto, en modelos como Conv1D, es especialmente relevante, debido a que su entrenamiento es estocástico, dependiendo mucho del azar procesos como la inicialización aleatoria de pesos, el orden de los *minibatches* o la aplicación del *dropout*; por lo que al promediar los resultados de las salidas obtenemos predicciones más estables. En el caso de los modelos de árboles, que son más deterministas, esto no tiene tanto efecto, ya que se fija una semilla para la aleatoriedad de los datos.

Por otra parte, el bagging inter-modelo, es decir, integrando los resultados de los diferentes modelos. Esto permite, como comentamos, complementar dos modelos de naturalezas diferentes, ya que por una parte capturamos relaciones no lineales de los datos y, por otra, patrones locales. Además, se han aplicado mecanismos específicos para controlar el sobreajuste:

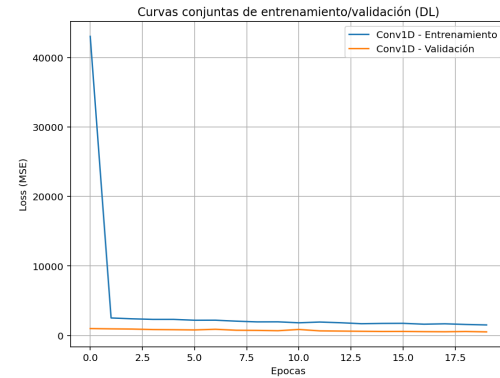
1. **EarlyStopping**: este mecanismo controla la evolución de la pérdida en el conjunto de validación, de forma que se detiene el entrenamiento si esta deja de mejorar durante un número determinado de épocas consecutivas, evitando que la red siga ajustándose en exceso a los datos de entrenamiento.
2. **ReduceLROnPlateau**: permite reducir automáticamente la tasa de aprendizaje si se detecta una estabilización de la pérdida, lo que facilita el refinamiento del aprendizaje y evita estancamientos.

Para validar los resultados del *bagging*, además de emplear las mismas métricas que se emplearon en los casos anteriores, analizaremos también las curvas de pérdida de entrenamiento y de validación, para Conv1D, ya que su ajuste progresivo de parámetros mediante retropropagación nos permite monitorizarlo. En este caso, tenemos la figura 11, que muestra en 11a las curvas para el horizonte de predicción semanal; y en 11b las curvas para el horizonte de predicción a cuatro semanas. La curva de entrenamiento representa el error del modelo al evaluarlo sobre los datos de train, y una disminución progresiva del mismo indica que el modelo aprende representaciones útiles de los datos. Por su parte, la curva de validación representa el error del modelo sobre datos no vistos (de test), y su evolución es crucial para analizar la capacidad de generalización. Estas curvas se analizan juntas, de forma que si ambas descienden de forma progresiva y se estabilizan en niveles bajos tendremos un modelo que generaliza bien. Si ambas permaneciesen altas, sin descender, tendríamos un caso de subajuste, donde el modelo no generaliza bien; si la de validación aumentara mientras se reduce la de entrenamiento, sería una señal inmediata de sobreajuste. Además, también podemos medir el sobreajuste viendo la distancia entre ambas curvas, siendo una distancia pequeña señal de que el modelo es bueno; mientras que una distancia grande indica *overfitting* (y, si se combina con valores muy elevados, indicaría *underfitting*).

En nuestro caso, vemos que la curva de entrenamiento desciende abruptamente en las primeras épocas para luego estabilizarse (pasa de 37000 a aproximadamente 2000, y se mantiene); mientras que la curva de validación, aunque se reduce, se mantiene estable desde el principio en un rango que no supera 1000. Esto indica que el modelo ha aprendido patrones relevantes de los datos (por la reducción significativa del *train loss*), a la vez que presenta buena capacidad de generalización (por la estabilidad en el *validation loss*) y los valores bajos que presenta. Aun así, podrían aplicarse técnicas adicionales para reducir aún más el *train loss* y su brecha respecto al *validation loss*.



(a) Curvas train-val loss para H=1



(b) Curvas train-val loss para H=1

Figura 11: Curvas train-val loss

Por último, analizaremos las métricas del modelo final de bagging (cuadro 3) y las compararemos con las que obtuvimos con los modelos base, tanto en la tabla 1 como en la tabla 2. En general, vemos que para el horizonte de predicción a corto plazo mejora los modelos, mientras que en horizontes más largos tiene un rendimiento más estable. En H=1, el R^2 aumentó ligeramente, desde el 0.9817 del Conv1D al 0.9882, pero con una destacable disminución de los errores, que se reducen a la mitad, pasando el RMSE de 30.50 a 16.26 y el MAE de 11.60 a 5.57. Para H=4, el R^2 se mantiene prácticamente estable, ya que únicamente baja de 0.9760 a 0.9757, y los errores apenas varían, pasando el RMSE de 35.32 a 34.87 y el MAE de 13.31 a 13.16. De esta forma, vemos que el bagging, en general, ofrece mayor robustez y estabilidad, aportando más ventajas en horizontes cortos, y manteniéndose más estable en horizontes largos.

Horizonte	MSE	RMSE	MAE	R2	MAPE	EVS
H1	264.543	16.2648	5.5755	0.9882	6.2485	0.9882
H4	1421.3568	34.8701	13.1624	0.9757	10.8791	0.9757

Cuadro 3: Resultados del modelo con bagging

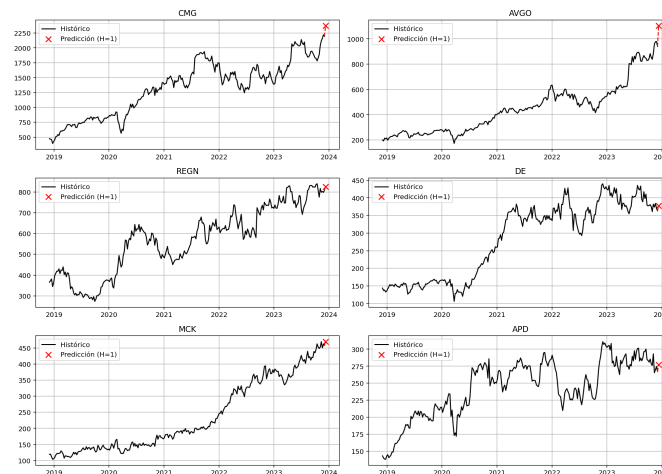


Figura 12: Predicciones para horizonte H=1

4. Resultados

Una vez aplicadas las distintas técnicas para obtener los mejores modelos, ya tenemos decidido aquel que usaremos para realizar las predicciones. Pero, si recordamos, como explicamos en 3.2.4, habíamos entrenado los modelos inicialmente sobre un subconjunto de los datos, con el objetivo de agilizar las pruebas, y poder entrenar únicamente el modelo final con todos los datos de todas las empresas. Una vez realizadas las predicciones, se obtienen los siguientes resultados.

```

1 Top 6 empresas con mayor crecimiento previsto (H=1):
2 CMG : Actual= 2194.45 | Pred= 2370.56 | DifAbs= 176.11 | Dif%= 8.03 %
3 AVGO: Actual= 940.53 | Pred= 1103.11 | DifAbs= 162.58 | Dif%= 17.29 %
4 REGN: Actual= 808.89 | Pred= 826.28 | DifAbs= 17.39 | Dif%= 2.15 %
5 DE : Actual= 363.31 | Pred= 378.08 | DifAbs= 14.77 | Dif%= 4.07 %
6 MCK : Actual= 458.51 | Pred= 468.86 | DifAbs= 10.35 | Dif%= 2.26 %
7 APD : Actual= 267.40 | Pred= 276.94 | DifAbs= 9.54 | Dif%= 3.57 %

```

En el caso de H=1, vemos que la empresa más rentable para invertir sería AVGO (Broadcom Inc.), ya que es la que proporciona una mayor crecimiento relativo, con un 17.29 % de incremento pronosticado. En lo que al crecimiento absoluto se refiere, la mejor empresa sería CMG (Chipotle Mexican Grill, Inc.), que nos daría un rendimiento de 176\$, pero tendríamos que invertir más dinero. En el gráfico 12 podemos ver estas seis empresas, graficadas su serie histórica y las predicciones en color rojo.

```

1 Top 6 empresas con mayor crecimiento previsto (H=4):
2 Empresa: ABEV. Ultimo valor: 2.74
3 +1 semana(s): Pred=4.83 | Dif=2.08 | Dif%=76.04 %
4 +2 semana(s): Pred=4.89 | Dif=2.14 | Dif%=78.22 %
5 +3 semana(s): Pred=5.03 | Dif=2.29 | Dif%=83.53 %
6 +4 semana(s): Pred=5.14 | Dif=2.40 | Dif%=87.47 %

```

7	
8	Empresa: MFG. Ultimo valor: 3.40
9	+1 semana(s): Pred=5.12 Dif=1.71 Dif %=50.34 %
10	+2 semana(s): Pred=5.11 Dif=1.70 Dif %=50.00 %
11	+3 semana(s): Pred=5.22 Dif=1.81 Dif %=53.21 %
12	+4 semana(s): Pred=5.33 Dif=1.92 Dif %=56.47 %
13	
14	Empresa: NOK. Ultimo valor: 3.58
15	+1 semana(s): Pred=5.16 Dif=1.58 Dif %=44.26 %
16	+2 semana(s): Pred=5.16 Dif=1.58 Dif %=44.01 %
17	+3 semana(s): Pred=5.27 Dif=1.69 Dif %=47.09 %
18	+4 semana(s): Pred=5.38 Dif=1.80 Dif %=50.28 %
19	
20	Empresa: TEF. Ultimo valor: 4.21
21	+1 semana(s): Pred=5.37 Dif=1.16 Dif %=27.67 %
22	+2 semana(s): Pred=5.38 Dif=1.17 Dif %=27.72 %
23	+3 semana(s): Pred=5.44 Dif=1.23 Dif %=29.31 %
24	+4 semana(s): Pred=5.56 Dif=1.35 Dif %=32.04 %
25	
26	Empresa: WIT. Ultimo valor: 4.78
27	+1 semana(s): Pred=5.92 Dif=1.14 Dif %=23.77 %
28	+2 semana(s): Pred=5.94 Dif=1.16 Dif %=24.34 %
29	+3 semana(s): Pred=6.00 Dif=1.22 Dif %=25.43 %
30	+4 semana(s): Pred=6.02 Dif=1.24 Dif %=25.92 %
31	
32	Empresa: NWG. Ultimo valor: 5.29
33	+1 semana(s): Pred=6.36 Dif=1.07 Dif %=20.21 %
34	+2 semana(s): Pred=6.37 Dif=1.08 Dif %=20.34 %
35	+3 semana(s): Pred=6.43 Dif=1.14 Dif %=21.55 %
36	+4 semana(s): Pred=6.40 Dif=1.11 Dif %=21.04 %

En el caso de $H=4$, observamos que la empresa más rentable para invertir sería ABEV (Ambev S.A.), ya que es la que muestra un mayor crecimiento relativo, con un 87.47 % de incremento pronosticado. Si atendemos al crecimiento absoluto, la mejor opción sería MFG (Mizuho Financial Group, Inc.), con un aumento proyectado de 1.92\$, aunque el importe de partida es menor que en otras compañías. En el gráfico 13 se representan estas seis empresas, mostrando en negro la serie histórica y en rojo las predicciones correspondientes a las cuatro semanas posteriores. Cabe destacar, además, que las predicciones para este muestran un crecimiento inicial más abrupto, estabilizándose en las semanas posteriores, lo que indica que no es el modelo más indicado.

Por último, analizamos el gráfico que resulta de realizar la predicción directamente con el horizonte $H=4$, frente a realizarla semana a semana, y retroalimentando el modelo con las nuevas predicciones. Podemos ver que las predicciones recursivas (en azul) tienden a capturar mejor fluctuaciones intermedias de los datos, aunque presenta más variabilidad e, indudablemente, es sensible a la acumulación de error conforme avanza el horizonte; mientras que las predicciones directas (en rojo) generan trayectorias más suaves, aunque con un sesgo o crecimiento inicial más abrupto. Como era de esperar, el modelo recursivo genera muy bien movimientos a corto plazo, mientras que el directo capta mejor las tendencias generales.



Figura 13: Predicciones para horizonte H=4

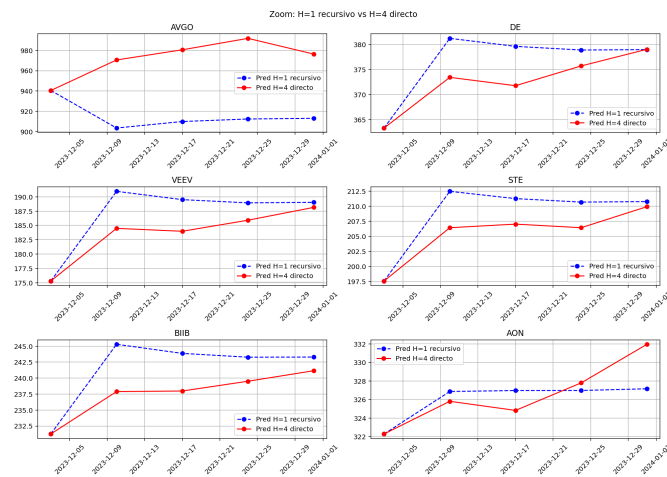


Figura 14: Comparación de predicciones para horizontes H=1 y H=4

5. Conclusiones

En este trabajo se ha desarrollado un algoritmo de Machine Learning para predecir series temporales financieras, aplicando diferentes modelos de aprendizaje automático y profundo: XGBoost, LightGBM, LSTM, GRU y Conv1D, con horizontes de predicción de una semana ($H=1$) y un mes ($H=4$). El objetivo del mismo era determinar en qué empresa sería más rentable invertir.

Se realizó un análisis inicial de los datos y un proceso de limpieza de los mismos, garantizando series consistentes, sin valores faltantes, y construyendo datasets supervisados con ventanas deslizantes. Posteriormente, se crearon y entrenaron los modelos, aplicando validación cruzada temporal para evaluar el rendimiento de los modelos evitando fugas de información.

Los resultados revelaron que el modelo convolucional ofreció las mejores métricas, logrando capturar muy bien los patrones locales de las series, seguido de los modelos basados en árboles de decisión (XGboost y LightGBM), mientras que los modelos de arquitecturas recurrentes (LSTM y GRU) mostraron un rendimiento inferior al esperado, probablemente por la complejidad de sus parámetros y la naturaleza de los datos. La incorporación de la técnica de *bagging* permitió incrementar, aún más, el rendimiento de los modelos, especialmente en horizontes temporales cortos.

Como ampliaciones o mejoras futuras de este trabajo, además de la puesta en producción del modelo en casos reales, el seguir ampliando los datos de entrenamiento semanalmente, conforme se vayan produciendo, así como ampliar los datos históricos previos. También se podrían añadir a los modelos variables exógenas, como indicadores macroeconómicos o noticias financieras, para enriquecer el contexto del modelo. También resultaría interesante poder explorar opciones más avanzadas, como transformers o redes convolucionales temporales (TCN), o probar con modelos híbridos, que combinen las fortalezas de las RNN y las CNN.

6. Bibliografía

1. Scikit-learn: <https://scikit-learn.org/stable/>
2. XGBoost: <https://xgboost.readthedocs.io/>
3. LightGBM: <https://lightgbm.readthedocs.io/>
4. TensorFlow Keras: <https://keras.io/>
5. Hyndman & Athanasopoulos, *Forecasting: Principles and Practice*: <https://otexts.com/fpp3/>
6. Machine Learning Mastery, explicación del bagging: <https://machinelearningmastery.com/bagging-and-random-forest-ensemble-algorithms-for-machine-learning/>
7. Validación cruzada en series temporales: https://scikit-learn.org/stable/modules/cross_validation.html#time-series-split

A. Código EDA

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from statsmodels.tsa.seasonal import seasonal_decompose
6 from statsmodels.stats.diagnostic import acorr_ljungbox
7
8 max_nan_ratio = 0.4
9 path_csv = "stock_details_5_years.csv"
10 export_path = "stock_weekly_clean.csv"
11
12 # ----- MAIN -----
13 if __name__ == "__main__":
14     path_csv = "stock_details_5_years.csv"
15
16     df = pd.read_csv(path_csv)
17     df['Date'] = pd.to_datetime(df['Date'], utc=True)
18
19     # ===== EDA preliminar antes del pivotado =====
20     print("\n=== EDA RAW (datos crudos) ===")
21     print(f"Empresas totales: {df['Company'].nunique()}")
22     print(f"Rango de fechas: {df['Date'].min().date()} a {df['Date'].max().date()}")
23
24     # Conteo por empresa
25     counts = df.groupby("Company")['Date'].count().sort_values(ascending=False)
26     print("\nTop 10 empresas con más registros:")
27     print(counts.head(10))
28     print("\nBottom 10 empresas con menos registros:")
29     print(counts.tail(10))
30
31     # ===== EDA sobre la matriz pivotada + limpieza + exportación =====
32     # Resample semanal
33     df_pivot = df.pivot(index='Date', columns='Company', values='Close').resample('W').last()
34
35     print("\n=== EDA Pivot (serie temporal por empresa) ===")
36     print(f"Dimensiones: {df_pivot.shape}")
37     print(f"Rango de fechas: {df_pivot.index.min().date()} a {df_pivot.index.max().date()}")
38
39     # Missing por empresa
40     na_ratio = df_pivot.isna().mean().sort_values(ascending=False)
41     na_ratio_row = na_ratio.to_frame().T
42     print("\n--- Porcentaje de missing por empresa (Top 10) ---")
43     print(na_ratio.head(10))

```

```
44
45 # Missing por fecha (tras eliminar empresas malas)
46 missing_by_date = df_pivot.isna().sum(axis=1)
47 plt.figure(figsize=(12, 4))
48 missing_by_date.plot()
49 plt.title("Número de valores faltantes por fecha (pre-limpieza)")
50 plt.xlabel("Fecha")
51 plt.ylabel("Num. de missing")
52 plt.show()
53
54 # Empresas eliminadas por exceso de NaN
55 dropped = na_ratio[na_ratio > max_nan_ratio]
56 if not dropped.empty:
57     print(f"\nEmpresas eliminadas (> {max_nan_ratio*100:.0f}% NaN): {
58         list(dropped.index)}")
59     df_pivot = df_pivot.drop(columns=dropped.index)
60
61 # Missing por fecha (tras eliminar empresas malas)
62 missing_by_date = df_pivot.isna().sum(axis=1)
63 plt.figure(figsize=(12, 4))
64 missing_by_date.plot()
65 plt.title("Número de valores faltantes por fecha (tras limpieza)")
66 plt.xlabel("Fecha")
67 plt.ylabel("Num. de missing")
68 plt.show()
69
70 # Imputación: interpolación temporal + ffill + bfill
71 df_pivot = df_pivot.interpolate(method='time').bfill()
72 print(f"Valores missing totales: {df_pivot.isna().sum().sum()}")
73
74 # Estadísticas
75 print("\n--- Estadísticas descriptivas (primeras 10 empresas) ---")
76 descr = df_pivot.describe().T
77 print(descr.head(10))
78 print("\n--- Estadísticas descriptivas (todas las empresas) ---")
79 print(descr)
80
81 # Chequeo de estacionalidad
82 print("\n--- Chequeo de estacionalidad ---")
83 seasonal_strength = {}
84 for company in df_pivot.columns:
85     try:
86         series = df_pivot[company].dropna()
87         decomposition = seasonal_decompose(series, period=52)
88         strength = decomposition.seasonal.var() / (series.var() + 1e-6)
89         seasonal_strength[company] = strength
90     except:
91         seasonal_strength[company] = np.nan
```

```

92     seasonal_strength_series = pd.Series(seasonal_strength).sort_values(
93         ascending=False)
94
95     # Top 20 en fuerza estacional
96     plt.figure(figsize=(12, 5))
97     sns.barplot(x=seasonal_strength_series.index[:20], y=
98         seasonal_strength_series.values[:20], color="steelblue")
99     plt.xticks(rotation=90)
100    plt.ylabel("Fuerza estacionalidad")
101    plt.title("Top 20 empresas por fuerza estacional (periodo=52 semanas)
102    ")
103    plt.show()
104
105    # Descomposición de 3 ejemplos: mayor, menor y mediana fuerza
106    estacional
107    examples = [
108        seasonal_strength_series.dropna().idxmax(),
109        seasonal_strength_series.dropna().idxmin(),
110        seasonal_strength_series.dropna().index[len(
111            seasonal_strength_series)//2]
112    ]
113    print(f"\nEjemplos de descomposición estacional (estacionalidad má
114    xima, mínima y mediana): {examples}")
115    for ex in examples:
116        series = df_pivot[ex].dropna()
117        decomposition = seasonal_decompose(series, period=52)
118        decomposition.plot()
119        plt.show()
120
121        residual = decomposition.resid
122        ljung_box_result = acorr_ljungbox(residual.dropna(), lags=[40],
123            return_df=True)
124        print(f"\nResultado de la prueba de Ljung-Box para la serie {ex}:
125            {ljung_box_result['lb_pvalue'].iloc[0]}")
126
127    # Gráfico comparativo de las 3 series
128    plt.figure(figsize=(12, 5))
129    for ex in examples:
130        plt.plot(df_pivot.index, df_pivot[ex], label=ex)
131    plt.legend()
132    plt.title("Comparación de series: mayor, menor y mediana
133    estacionalidad")
134    plt.grid(True)
135    plt.show()
136
137    # Gráfico de todas las empresas
138    plt.figure(figsize=(12, 5))
139    for company in df_pivot.columns:
140        plt.plot(df_pivot.index, df_pivot[company], label=company)
141    #plt.legend()
142    plt.title("Gráfico de todas las empresas (tras limpieza)")

```

```
134     plt.grid(True)
135     plt.show()
136
137     # Exportar dataset limpio
138     df_pivot.to_csv(export_path)
139     print(f"\nDataset limpio exportado a: {export_path}")
```

B. Código para H=4

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.metrics import mean_squared_error, r2_score,
  mean_absolute_error, explained_variance_score
5 from sklearn.model_selection import ParameterGrid, TimeSeriesSplit
6 from xgboost import XGBRegressor
7 from lightgbm import LGBMRegressor
8 from tensorflow.keras.models import Sequential
9 from tensorflow.keras.layers import LSTM, GRU, Conv1D, Flatten, Dropout,
  Dense
10 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
11 from itertools import product
12 from sklearn.base import clone
13 import matplotlib.pyplot as plt
14
15 import warnings
16 warnings.filterwarnings("ignore")
17
18 SEED = 42
19
20 # ----- CREATE SUPERVISED WITH DATES (H=1) -----
21 def create_supervised_with_dates(df, W=25, H=1):
22     X, y, dates = [], [], []
23     for col in df.columns:
24         data = df[col].values
25         idx = df.index
26         for i in range(len(data) - W - H + 1):
27             X.append(data[i:i+W])
28             # H=1 -> sacamos un escalar, pero lo guardamos como vector (n
29             # ,1) para Keras/consistencia
30             y.append([data[i+W]])
31             dates.append([idx[i+W]]) # fecha objetivo (una semana
32             siguiente)
33     return np.array(X), np.array(y), np.array(dates)
34
35 # ----- METRICS -----
36 def compute_metrics(y_true, y_pred):
37     # y_true/pred pueden venir con shape (n,1) -> a 1D
38     y_true = np.ravel(y_true)
39     y_pred = np.ravel(y_pred)
40     mse = mean_squared_error(y_true, y_pred)
41     rmse = np.sqrt(mse)
42     mae = mean_absolute_error(y_true, y_pred)
43     r2 = r2_score(y_true, y_pred)
44     mape = np.mean(np.abs((y_true - y_pred)/(y_true+1e-6))) * 100
45     evs = explained_variance_score(y_true, y_pred)

```

```

44     return {"MSE":mse, "RMSE":rmse, "MAE":mae, "R2":r2, "MAPE":mape, "EVS":evs
45     }
46
47 # ----- DL BUILDERS (H=1) -----
48 def build_lstm(input_shape, units=16, dropout=0.3, H=1, **kwargs):
49     model = Sequential([
50         LSTM(units, input_shape=input_shape),
51         Dropout(dropout),
52         Dense(H)
53     ])
54     model.compile(optimizer='adam', loss='mse')
55     return model
56
57 def build_gru(input_shape, units=16, dropout=0.3, H=1, **kwargs):
58     model = Sequential([
59         GRU(units, input_shape=input_shape),
60         Dropout(dropout),
61         Dense(H)
62     ])
63     model.compile(optimizer='adam', loss='mse')
64     return model
65
66 def build_conv1d(input_shape, units=16, dropout=0.3, kernel_size=3, H=1,
67 **kwargs):
68     model = Sequential([
69         Conv1D(filters=units, kernel_size=kernel_size, activation='relu',
70             input_shape=input_shape),
71         Flatten(),
72         Dropout(dropout),
73         Dense(H)
74     ])
75     model.compile(optimizer='adam', loss='mse')
76     return model
77
78 # ----- TSCV: ML -----
79 def tscv_ml(model, X, y, n_splits):
80     # y shape -> (n,1) o (n,) -> a 1D
81     y = np.ravel(y)
82     tscv = TimeSeriesSplit(n_splits=n_splits)
83     metrics_list = []
84     X_flat = X.reshape((X.shape[0], -1))
85     for train_idx, test_idx in tscv.split(X_flat):
86         X_train, X_test = X_flat[train_idx], X_flat[test_idx]
87         y_train, y_test = y[train_idx], y[test_idx]
88         scaler = StandardScaler()
89         X_train_scaled = scaler.fit_transform(X_train)
90         X_test_scaled = scaler.transform(X_test)
91         model_fold = clone(model)
92         model_fold.fit(X_train_scaled, y_train)
93         y_pred = model_fold.predict(X_test_scaled)
94         metrics_list.append(compute_metrics(y_test, y_pred))

```

```

92     return {k: np.mean([m[k] for m in metrics_list]) for k in
93            metrics_list[0]}
94
95 # ----- TSCV: DL -----
96 def tscv_dl(model_builder, X, y, n_splits, units, dropout, epochs,
97            batch_size, kernel_size=3):
98     tscv = TimeSeriesSplit(n_splits=n_splits)
99     metrics_list = []
100     # X_seq: (n, W, 1)
101     X_seq = X.reshape((X.shape[0], X.shape[1], 1))
102     # y: (n, 1)
103     y = y.reshape(-1, 1)
104
105     for train_idx, test_idx in tscv.split(X_seq):
106         X_train, X_test = X_seq[train_idx], X_seq[test_idx]
107         y_train, y_test = y[train_idx], y[test_idx]
108
109         # Estandarizamos X por ventana (aplanamos y re-formamos)
110         scaler = StandardScaler()
111         X_train_2d = X_train.reshape((X_train.shape[0], -1))
112         X_test_2d = X_test.reshape((X_test.shape[0], -1))
113         X_train_scaled = scaler.fit_transform(X_train_2d).reshape(X_train
114            .shape)
115         X_test_scaled = scaler.transform(X_test_2d).reshape(X_test.shape)
116
117         model = model_builder(input_shape=(X_train.shape[1], 1),
118                               units=units, dropout=dropout, H=1,
119                               kernel_size=kernel_size)
120         es = EarlyStopping(monitor='val_loss', patience=2,
121                             restore_best_weights=True)
122         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
123                                =1)
124         model.fit(X_train_scaled, y_train, validation_data=(X_test_scaled
125            , y_test),
126                  epochs=epochs, batch_size=batch_size, verbose=0,
127                  callbacks=[es, lr])
128         y_pred = model.predict(X_test_scaled, verbose=0)
129         metrics_list.append(compute_metrics(y_test, y_pred))
130
131     return {k: np.mean([m[k] for m in metrics_list]) for k in
132            metrics_list[0]}
133
134 # ----- Funciones gráficas -----
135 def plot_learning_curves(histories, title="Curvas de entrenamiento/
136 validación"):
137     """
138     histories: puede ser
139         - un único objeto History de Keras
140         - un diccionario {nombre: history} para graficar varios modelos
141         juntos
142     """

```

```

132 plt.figure(figsize=(8,6))
133
134 if isinstance(histories, dict):
135     # Varios modelos
136     for name, hist in histories.items():
137         plt.plot(hist.history['loss'], label=f'{name} - Entrenamiento')
138         if 'val_loss' in hist.history:
139             plt.plot(hist.history['val_loss'], label=f'{name} - Validación')
140 else:
141     # Un solo modelo
142     hist = histories
143     plt.plot(hist.history['loss'], label='Entrenamiento')
144     if 'val_loss' in hist.history:
145         plt.plot(hist.history['val_loss'], label='Validación')
146
147 plt.xlabel("Epocas")
148 plt.ylabel("Loss (MSE)")
149 plt.title(title)
150 plt.legend()
151 plt.grid(True)
152 plt.show()
153
154 # ----- MAIN PIPELINE -----
155 if __name__=="__main__":
156     path_csv = "stock_weekly_clean.csv"
157     W, H = 25, 1 # horizonte simplificado a 1 semana
158     sub_sample_ratio = 0.4
159     np.random.seed(SEED)
160
161     # ---- Datos semanales por empresa
162     df_weekly = pd.read_csv(path_csv, index_col=0, parse_dates=True)
163     df_weekly.index = pd.to_datetime(df_weekly.index, utc=True)
164     sampled_cols = np.random.choice(df_weekly.columns, int(len(df_weekly.columns)*sub_sample_ratio), replace=False)
165     df_sub = df_weekly[sampled_cols]
166
167     # ---- Supervised
168     X_sub, y_sub, _ = create_supervised_with_dates(df_sub, W=W, H=H)
169     X_sub = np.nan_to_num(X_sub)
170     y_sub = np.nan_to_num(y_sub) # (n,1)
171
172     # ----- HYPERPARAMETERS -----
173     xgb_params = {"n_estimators": [100,200], "max_depth": [3,5], "learning_rate": [0.01,0.05]}
174     lgb_params = {"n_estimators": [100,200], "max_depth": [5,10], "learning_rate": [0.01,0.05]}
175     dl_params = {'units': [16,32,64], 'dropout': [0.2,0.3]}
176     c1d_params = {'units': [16,32,64], 'dropout': [0.2,0.3], 'kernel_size': [3,5]}

```



```

177
178     best_models = {}
179     tuning_results = {}
180     results_sub = {}
181
182     # ----- ML MODELS (tuning con TSCV) -----
183     ml_models = {"XGB": XGBRegressor, "LGBM": LGBMRegressor}
184     ml_param_grids = {"XGB": xgb_params, "LGBM": lgb_params}
185
186     for name, ModelClass in ml_models.items():
187         best_score, best_model = -np.inf, None
188         for params in ParameterGrid(ml_param_grids[name]):
189             print(f"{name} con {params}")
190             if name=="XGB":
191                 model = ModelClass(random_state=SEED, verbosity=0, **
192                                     params)
193             else:
194                 model = ModelClass(random_state=SEED, verbose=-1, **
195                                     params)
196             metrics = tscv_ml(model, X_sub, y_sub, n_splits=5)
197             tuning_results.setdefault(name, []).append({'params': params,
198                                                         'metrics': metrics})
199             if metrics['R2'] > best_score:
200                 best_score = metrics['R2']
201                 best_model = model
202             # Guardamos mejores
203             results_sub[name] = tscv_ml(best_model, X_sub, y_sub, n_splits=5)
204             best_models[name] = best_model
205
206     # ----- DL MODELS (tuning con TSCV) -----
207     dl_builders = {"LSTM": build_lstm, "GRU": build_gru, "Conv1D":
208                    build_conv1d}
209     dl_param_grids = {"LSTM": dl_params, "GRU": dl_params, "Conv1D":
210                       cid_params}
211
212     for name, builder in dl_builders.items():
213         best_score, best_config, best_metrics = -np.inf, None, None
214         param_grid = dl_param_grids[name]
215
216         if name == "Conv1D":
217             combos = product(param_grid['units'], param_grid['dropout'],
218                             param_grid['kernel_size'])
219         else:
220             combos = product(param_grid['units'], param_grid['dropout'])
221
222         for combo in combos:
223             print(f"{name} con {combo}")
224             if name == "Conv1D":
225                 units, dropout, kernel_size = combo
226             metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,

```

```

221         units=units, dropout=dropout, epochs
222         =10, batch_size=32,
223         kernel_size=kernel_size)
224     config = {'units': units, 'dropout': dropout, '
225             kernel_size': kernel_size}
226     else:
227         units, dropout = combo
228         metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
229                           units=units, dropout=dropout, epochs
230                           =10, batch_size=32)
231         config = {'units': units, 'dropout': dropout}
232
233     tuning_results.setdefault(name, []).append({'params': config,
234         'metrics': metrics})
235
236     if metrics['R2'] > best_score:
237         best_score = metrics['R2']
238         best_config = config
239         best_metrics = metrics
240
241     # Guardamos todas las métricas del mejor modelo + su config
242     results_sub[name] = best_metrics
243     results_sub[name]['config'] = best_config
244
245     # Guardamos config para reentrenar más adelante
246     best_models[name] = best_config
247
248     print("\n=== Métricas de los mejores modelos (submuestra) ===")
249     pd.set_option("display.max_rows", None)
250     pd.set_option("display.max_columns", None)
251     pd.set_option("display.width", None)
252     pd.set_option("display.max_colwidth", None)
253     df_results = pd.DataFrame(results_sub).T
254     print(df_results.sort_values("R2", ascending=False))
255
256     # ----- SELECT BEST MODEL (solo reporte) -----
257     best_r2 = -np.inf
258     best_model_name = None
259     for name, m in best_models.items():
260         if name in ["XGB", "LGBM"]:
261             r2 = tscv_ml(m, X_sub, y_sub, n_splits=5)['R2']
262         else:
263             r2 = tscv_dl(dl_builders[name], X_sub, y_sub, n_splits=3, **m,
264                           epochs=10, batch_size=32)['R2']
265         if r2 > best_r2:
266             best_r2 = r2
267             best_model_name = name
268     print(f"Mejor modelo por R2: {best_model_name} (R2={best_r2:.4f})")
269
270     # ----- PREPARAR DATOS COMPLETOS -----

```

```

266 X_full, y_full, dates_full = create_supervised_with_dates(df_sub, W=W
267 , H=H)
268 X_full = np.nan_to_num(X_full)
269 y_full = np.nan_to_num(y_full) # (n,1)
270 scaler_full = StandardScaler()
271 X_full_scaled = scaler_full.fit_transform(X_full)
272
273 # ----- BAGGING GLOBAL (último fold, validación)
274 -----
275 print("=== Bagging (validación, último fold) ===")
276 top_models = ["LGBM", "Conv1D"]
277 n_bags = 5
278
279 tscv = TimeSeriesSplit(n_splits=5)
280 train_idx, test_idx = list(tscv.split(X_full_scaled))[-1]
281 X_train, X_test = X_full_scaled[train_idx], X_full_scaled[test_idx]
282 y_train, y_test = y_full[train_idx], y_full[test_idx] # (n,1)
283
284 bagging_preds = np.zeros(y_test.shape[0], dtype=float)
285 histories = {}
286
287 for name in top_models:
288     fold_preds = np.zeros(y_test.shape[0], dtype=float)
289     for _ in range(n_bags):
290         if name in ["XGB", "LGBM"]:
291             base = best_models[name] # es un modelo ya configurado
292             model = clone(base)
293             model.fit(X_train, np.ravel(y_train))
294             fold_preds += model.predict(X_test)
295         else:
296             # DL
297             config = best_models[name]
298             X_train_seq = X_train.reshape((X_train.shape[0], W, 1))
299             X_test_seq = X_test.reshape((X_test.shape[0], W, 1))
300             model = dl_builders[name](input_shape=(W,1), H=1, **
301 config)
302             es = EarlyStopping(monitor='val_loss', patience=3,
303 restore_best_weights=True)
304             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
305 patience=2)
306             history = model.fit(X_train_seq, y_train, validation_data
307 =(X_test_seq, y_test),
308 epochs=20, batch_size=32, verbose=0, callbacks
309 =[es, lr])
310             histories[name] = history
311             fold_preds += model.predict(X_test_seq, verbose=0).
312 flatten()
313
314 fold_preds /= n_bags
315 bagging_preds += fold_preds
316
317
318

```

```

309     bagging_preds /= len(top_models)
310     metrics = compute_metrics(y_test, bagging_preds)
311     plot_learning_curves(histories, title="Curvas conjuntas de
entrenamiento/validación (DL)")
312     print("Métricas Bagging (H=1):", {k: round(v,4) for k,v in metrics.
items()})
313
314     # ----- REENTRENAR EN TODOS LOS DATOS (para predicción
futura) -----
315     print("\n=== Reentrenamiento en todos los datos para predicción futura ==="
)
316     trained_models = {}
317     for name in top_models:
318         if name in ["XGB", "LGBM"]:
319             base = best_models[name]
320             model = clone(base)
321             model.fit(X_full_scaled, np.ravel(y_full))
322         else:
323             config = best_models[name]
324             X_full_seq = X_full_scaled.reshape((X_full_scaled.shape[0], W
, 1))
325             model = dl_builders[name](input_shape=(W,1), H=1, **config)
326             es = EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True)
327             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
patience=2)
328             model.fit(X_full_seq, y_full, epochs=20, batch_size=32,
verbose=0, callbacks=[es, lr])
329             trained_models[name] = model
330
331     # ----- PREDICCIÓN FUTURA (una semana) -----
332     print("\n=== Predicción futura (última ventana de cada empresa, H=1)
===")
333     n_companies = df_sub.shape[1]
334     y_pred_last = np.zeros(n_companies)
335
336     for i, company in enumerate(df_sub.columns):
337         series = df_sub[company].values
338         last_window = series[-W:].reshape(1, -1)
339         last_window_scaled = scaler_full.transform(last_window)
340
341         preds_models = []
342         for name, model in trained_models.items():
343             if name in ["XGB", "LGBM"]:
344                 preds_models.append(float(model.predict(
last_window_scaled)[0]))
345             else:
346                 last_seq = last_window_scaled.reshape((1, W, 1))
347                 preds_models.append(float(model.predict(last_seq, verbose
=0).flatten()[0]))
348         y_pred_last[i] = np.mean(preds_models) # bagging entre modelos

```

```

349
350 # ----- TOP 6 + LISTA -----
351 last_values = df_sub.iloc[-1].values
352 diff_abs = y_pred_last - last_values
353 diff_pct = 100 * diff_abs / (last_values + 1e-6)
354 top6_idx = np.argsort(diff_abs)[-6:][::-1]
355
356 print("\nTop 6 empresas con mayor crecimiento previsto (H=1):")
357 for idx in top6_idx:
358     print(f"{df_sub.columns[idx]:<4}: Actual={last_values[idx]:>8.2f}
359           | Pred={y_pred_last[idx]:>8.2f} | "
360           f"DifAbs={diff_abs[idx]:>7.2f} | Dif%={diff_pct[idx]:>6.2f}%")
361
362 # ----- GRAFICO 3x2 (Histórico + punto de predicción)
363 -----
364 fig, axes = plt.subplots(3,2, figsize=(14,10))
365 axes = axes.flatten()
366 last_date = df_sub.index[-1]
367 next_date = last_date + pd.Timedelta(weeks=1)
368
369 for j, idx in enumerate(top6_idx):
370     company = df_sub.columns[idx]
371
372     # Histórico
373     axes[j].plot(df_sub.index, df_sub[company].values, label="Histó
374     rico", color="black")
375
376     # Predicción como cruz roja
377     axes[j].scatter([next_date], [y_pred_last[idx]], color="red",
378     label="Predicción (H=1)", s=70, marker="x")
379
380     # Línea roja que conecta último valor real con la predicción
381     axes[j].plot([last_date, next_date], [last_values[idx],
382     y_pred_last[idx]], color="red", linestyle="--")
383
384     axes[j].set_title(company)
385     axes[j].legend()
386     axes[j].grid(True)
387
388 plt.tight_layout()
389 plt.show()

```

C. Código para H=1

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.metrics import mean_squared_error, r2_score,
  mean_absolute_error, explained_variance_score
5 from sklearn.multioutput import MultiOutputRegressor
6 from sklearn.model_selection import ParameterGrid, TimeSeriesSplit
7 from xgboost import XGBRegressor
8 from lightgbm import LGBMRegressor
9 from tensorflow.keras.models import Sequential
10 from tensorflow.keras.layers import LSTM, GRU, Conv1D, Flatten, Dropout,
  Dense
11 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
12 from itertools import product
13 from sklearn.base import clone
14 import matplotlib.pyplot as plt
15
16 import warnings
17 warnings.filterwarnings("ignore")
18
19 SEED = 42
20
21 # ----- CREATE SUPERVISED WITH DATES -----
22 def create_supervised_with_dates(df, W=25, H=4):
23     X, y, dates = [], [], []
24     for col in df.columns:
25         data = df[col].values
26         idx = df.index
27         for i in range(len(data)-W-H+1):
28             X.append(data[i:i+W])
29             y.append(data[i+W:i+W+H])
30             dates.append(idx[i+W:i+W+H])
31     return np.array(X), np.array(y), np.array(dates)
32
33 # ----- METRICS -----
34 def compute_metrics(y_true, y_pred):
35     mse = mean_squared_error(y_true, y_pred)
36     rmse = np.sqrt(mse)
37     mae = mean_absolute_error(y_true, y_pred)
38     r2 = r2_score(y_true, y_pred)
39     mape = np.mean(np.abs((y_true - y_pred)/(y_true+1e-6))) * 100
40     da = np.mean((y_true>0) == (y_pred>0))
41     smape = 100*np.mean(2*np.abs(y_pred-y_true)/(np.abs(y_true)+np.abs(
  y_pred)+1e-6))
42     evs = explained_variance_score(y_true, y_pred)
43     return {"MSE":mse, "RMSE":rmse, "MAE":mae, "R2":r2, "MAPE":mape, "EVS":evs
  }
44

```

```

45 # ----- DL MODELS -----
46 def build_lstm(input_shape, units=16, dropout=0.3, H=4, **kwargs):
47     model = Sequential([LSTM(units, input_shape=input_shape),
48                         Dropout(dropout),
49                         Dense(H)])
50     model.compile(optimizer='adam', loss='mse')
51     return model
52
53 def build_gru(input_shape, units=16, dropout=0.3, H=4, **kwargs):
54     model = Sequential([GRU(units, input_shape=input_shape),
55                         Dropout(dropout),
56                         Dense(H)])
57     model.compile(optimizer='adam', loss='mse')
58     return model
59
60 def build_conv1d(input_shape, units=16, dropout=0.3, kernel_size=3, H=4,
61 **kwargs):
62     model = Sequential([Conv1D(filters=units, kernel_size=kernel_size,
63                                activation='relu', input_shape=input_shape),
64                         Flatten(),
65                         Dropout(dropout),
66                         Dense(H)])
67     model.compile(optimizer='adam', loss='mse')
68     return model
69
70 # ----- ML con TimeSeriesSplit -----
71 def tscv_ml(model, X, y, n_splits):
72     tscv = TimeSeriesSplit(n_splits=n_splits)
73     metrics_list = []
74     X_flat = X.reshape((X.shape[0], -1))
75     for train_idx, test_idx in tscv.split(X_flat):
76         X_train, X_test = X_flat[train_idx], X_flat[test_idx]
77         y_train, y_test = y[train_idx], y[test_idx]
78         scaler = StandardScaler()
79         X_train_scaled = scaler.fit_transform(X_train)
80         X_test_scaled = scaler.transform(X_test)
81         model_fold = MultiOutputRegressor(clone(model)) if y.shape[1]>1
82         else clone(model)
83         model_fold.fit(X_train_scaled, y_train)
84         y_pred = model_fold.predict(X_test_scaled)
85         metrics_list.append(compute_metrics(y_test, y_pred))
86     return {k: np.mean([m[k] for m in metrics_list]) for k in
87             metrics_list[0]}
88
89 # ----- DL con TimeSeriesSplit -----
90 def tscv_dl(model_builder, X, y, n_splits, units, dropout, epochs,
91             batch_size, kernel_size=3):
92     tscv = TimeSeriesSplit(n_splits=n_splits)
93     metrics_list = []
94     X_seq = X.reshape((X.shape[0], X.shape[1], 1))

```

```

91     for train_idx, test_idx in tscv.split(X_seq):
92         X_train, X_test = X_seq[train_idx], X_seq[test_idx]
93         y_train, y_test = y[train_idx], y[test_idx]
94         scaler = StandardScaler()
95         X_train_2d = X_train.reshape((X_train.shape[0], -1))
96         X_test_2d = X_test.reshape((X_test.shape[0], -1))
97         X_train_scaled = scaler.fit_transform(X_train_2d).reshape(X_train
98         .shape)
99         X_test_scaled = scaler.transform(X_test_2d).reshape(X_test.shape)
100
101         model = model_builder(input_shape=(X_train.shape[1],1),
102                                units=units, dropout=dropout, H=y_train.
103                                shape[1], kernel_size=kernel_size)
104         es = EarlyStopping(monitor='val_loss', patience=2,
105                             restore_best_weights=True)
106         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
107                                =1)
108         model.fit(X_train_scaled, y_train, validation_data=(X_test_scaled
109         , y_test),
110                   epochs=epochs, batch_size=batch_size, verbose=0,
111                   callbacks=[es, lr])
112         y_pred = model.predict(X_test_scaled, verbose=0)
113         metrics_list.append(compute_metrics(y_test, y_pred))
114
115     return {k: np.mean([m[k] for m in metrics_list]) for k in
116            metrics_list[0]}
117
118 # ----- Funciones gráficas -----
119 def plot_learning_curves(histories, title="Curvas de entrenamiento/
120 validación"):
121     """
122     histories: puede ser
123         - un único objeto History de Keras
124         - un diccionario {nombre: history} para graficar varios modelos
125         juntos
126     """
127     plt.figure(figsize=(8,6))
128
129     if isinstance(histories, dict):
130         # Varios modelos
131         for name, hist in histories.items():
132             plt.plot(hist.history['loss'], label=f'{name} - Entrenamiento
133             ')
134             if 'val_loss' in hist.history:
135                 plt.plot(hist.history['val_loss'], label=f'{name} -
136                 Validación')
137     else:
138         # Un solo modelo
139         hist = histories
140         plt.plot(hist.history['loss'], label='Entrenamiento')
141         if 'val_loss' in hist.history:

```



```

131         plt.plot(hist.history['val_loss'], label='Validación')
132
133     plt.xlabel("Epocas")
134     plt.ylabel("Loss (MSE)")
135     plt.title(title)
136     plt.legend()
137     plt.grid(True)
138     plt.show()
139
140     # ----- MAIN PIPELINE -----
141     if __name__ == "__main__":
142         path_csv = "stock_weekly_clean.csv"
143         W,H = 25,4
144         sub_sample_ratio = 0.4
145         np.random.seed(SEED)
146
147         df_weekly = pd.read_csv(path_csv, index_col=0, parse_dates=True)
148         df_weekly.index = pd.to_datetime(df_weekly.index, utc=True)
149         sampled_cols = np.random.choice(df_weekly.columns, int(len(df_weekly.
150             columns)*sub_sample_ratio), replace=False)
151         df_sub = df_weekly[sampled_cols]
152
153         X_sub, y_sub, _ = create_supervised_with_dates(df_sub, W=W, H=H)
154         X_sub = np.nan_to_num(X_sub)
155         y_sub = np.nan_to_num(y_sub)
156
157         # ----- HYPERPARAMETERS -----
158         xgb_params = {"n_estimators": [100, 200], "max_depth": [3, 5], "
159             learning_rate": [0.01, 0.05]}
160         lgb_params = {"n_estimators": [100, 200], "max_depth": [5, 10], "
161             learning_rate": [0.01, 0.05]}
162         dl_params = {'units': [16, 32, 64], 'dropout': [0.2, 0.3]}
163         cid_params = {'units': [16, 32, 64], 'dropout': [0.2, 0.3], 'kernel_size'
164             : [3, 5]}
165
166         best_models = {}
167         tuning_results = {}
168         results_sub = {}
169
170         # ----- ML MODELS -----
171         ml_models = {"XGB": XGBRegressor, "LGBM": LGBMRegressor}
172         ml_param_grids = {"XGB": xgb_params, "LGBM": lgb_params}
173
174         for name, ModelClass in ml_models.items():
175             best_score, best_model = -np.inf, None
176             for params in ParameterGrid(ml_param_grids[name]):
177                 print(f"{name} con {params}")
178                 if name == "XGB":
179                     model = ModelClass(random_state=SEED, verbosity=0, **
180                         params)
181                 else:

```

```

177         model = ModelClass(random_state=SEED, verbose=-1, **
178                               params)
179         metrics = tscv_ml(model, X_sub, y_sub, n_splits=5)
180         tuning_results.setdefault(name, []).append({'params': params,
181             'metrics': metrics})
182         if metrics['R2'] > best_score:
183             best_score = metrics['R2']
184             best_model = model
185         results_sub[name] = tscv_ml(best_model, X_sub, y_sub, n_splits=5)
186         best_models[name] = best_model
187
188     # ----- DL MODELS -----
189     dl_models = {"LSTM": build_lstm, "GRU": build_gru, "Conv1D":
190         build_conv1d}
191     dl_param_grids = {"LSTM": dl_params, "GRU": dl_params, "Conv1D":
192         cid_params}
193
194     for name, builder in dl_models.items():
195         best_score, best_config, best_metrics = -np.inf, None, None
196         param_grid = dl_param_grids[name]
197         combos = product(*(param_grid.values())) if name!="Conv1D" else
198             product(param_grid['units'], param_grid['dropout'], param_grid[
199                 'kernel_size'])
200         for combo in combos:
201             print(f"{name} con {combo}")
202             if name=="Conv1D":
203                 units, dropout, kernel_size = combo
204                 metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
205                     units=units, dropout=dropout,
206                         kernel_size=kernel_size, epochs=10,
207                         batch_size=32)
208                 config = {'units': units, 'dropout': dropout, '
209                     kernel_size': kernel_size}
210             else:
211                 units, dropout = combo
212                 metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
213                     units=units, dropout=dropout,
214                         epochs=10, batch_size=32, kernel_size
215                         =3)
216                 config = {'units': units, 'dropout': dropout}
217
218             tuning_results.setdefault(name, []).append({'params': config,
219                 'metrics': metrics})
220
221             if metrics['R2'] > best_score:
222                 best_score = metrics['R2']
223                 best_config = config
224                 best_metrics = metrics
225
226     # Guardamos todas las métricas del mejor modelo
227     results_sub[name] = (**best_metrics, 'config': best_config)

```

```

216         best_models[name] = best_config
217
218         # ----- SELECT BEST MODEL -----
219         best_r2 = -np.inf
220         best_model_name = None
221         for name, m in best_models.items():
222             if name in ["XGB", "LGBM"]:
223                 r2 = tscv_ml(m, X_sub, y_sub, n_splits=5)['R2']
224             else:
225                 r2 = tscv_dl(dl_models[name], X_sub, y_sub, n_splits=3, **m,
226                             epochs=10, batch_size=32)['R2']
227             if r2 > best_r2:
228                 best_r2 = r2
229                 best_model_name = name
230
231         print("\n=== Métricas de los mejores modelos (submuestra) ===")
232         pd.set_option("display.max_rows", None)
233         pd.set_option("display.max_columns", None)
234         pd.set_option("display.width", None)
235         pd.set_option("display.max_colwidth", None)
236         print(pd.DataFrame(results_sub).T)
237
238         top_models = ["LGBM", "Conv1D"]
239
240         # ----- BAGGING GLOBAL -----
241         print("=== Bagging simple por empresa ===")
242
243         n_bags = 5
244
245         # Preparamos datos completos supervisados
246         X_full, y_full, dates_full = create_supervised_with_dates(df_sub, W=W
247         , H=H)
248         X_full = np.nan_to_num(X_full)
249         y_full = np.nan_to_num(y_full)
250
251         # Escalamos
252         scaler_full = StandardScaler()
253         X_full_scaled = scaler_full.fit_transform(X_full)
254
255         # Ultimo fold train/test
256         tscv = TimeSeriesSplit(n_splits=5)
257         train_idx, test_idx = list(tscv.split(X_full_scaled))[-1]
258         X_train, X_test = X_full_scaled[train_idx], X_full_scaled[test_idx]
259         y_train, y_test = y_full[train_idx], y_full[test_idx]
260
261         # Bagging sobre todos los modelos
262         bagging_preds = np.zeros_like(y_test, dtype=float)
263         histories = {}
264
265         for name in top_models:
266             preds_bag = np.zeros_like(y_test, dtype=float)

```

```

265     for _ in range(n_bags):
266         if name in ["XGB", "LGBM"]:
267             model = MultiOutputRegressor(clone(best_models[name])) if
                y_train.shape[1] > 1 else clone(best_models[name])
268             model.fit(X_train, y_train)
269             preds_bag += model.predict(X_test)
270         else: # Conv1D
271             config = best_models[name]
272             X_train_seq = X_train.reshape((X_train.shape[0], W, 1))
273             X_test_seq = X_test.reshape((X_test.shape[0], W, 1))
274             model = dl_models[name](input_shape=(W, 1), H=y_train.
                shape[1], **config)
275             es = EarlyStopping(monitor='val_loss', patience=3,
                restore_best_weights=True)
276             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                patience=2)
277             history = model.fit(X_train_seq, y_train, validation_data
                =(X_test_seq, y_test),
278                             epochs=20, batch_size=32, verbose=0, callbacks
                =[es, lr])
279             histories[name] = history
280             preds_bag += model.predict(X_test_seq, verbose=0)
281         preds_bag /= n_bags
282         bagging_preds += preds_bag
283
284     bagging_preds /= len(top_models)
285     plot_learning_curves(histories, title="Curvas conjuntas de
entrenamiento/validación (DL)")
286     bagging_metrics = compute_metrics(y_test, bagging_preds)
287     print("Métricas Bagging (H=4):", {k: round(v,4) for k,v in metrics.
items()})
288
289     # ----- PREDICCIÓN FUTURA FINAL -----
290     print("\n=== Predicción futura (última ventana de cada empresa) ===")
291
292     n_companies = df_sub.shape[1]
293     y_pred_last = np.zeros((n_companies, H)) # Asegurar matriz 2D
294
295     # Entrenamos cada modelo una vez con todos los datos
296     trained_models = {}
297     for name in top_models:
298         if name in ["XGB", "LGBM"]:
299             model = MultiOutputRegressor(clone(best_models[name])) if H >
                1 else clone(best_models[name])
300             model.fit(X_full_scaled, y_full)
301         else:
302             config = best_models[name]
303             X_full_seq = X_full_scaled.reshape((X_full_scaled.shape[0], W
                , 1))
304             model = dl_models[name](input_shape=(W,1), H=H, **config)

```

```

305         es = EarlyStopping(monitor='val_loss', patience=3,
306                             restore_best_weights=True)
307         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
308                                patience=2)
309         model.fit(X_full_seq, y_full, epochs=20, batch_size=32,
310                  verbose=0, callbacks=[es, lr])
311         trained_models[name] = model
312
313     # Predicción empresa por empresa
314     for i, company in enumerate(df_sub.columns):
315         series = df_sub[company].values
316         last_window = series[-W:].reshape(1, -1)
317         last_window_scaled = scaler_full.transform(last_window)
318
319         preds_models = []
320         for name, model in trained_models.items():
321             if name in ["XGB", "LGBM"]:
322                 pred = model.predict(last_window_scaled)
323             else: # DL
324                 last_seq = last_window_scaled.reshape((1,W,1))
325                 pred = model.predict(last_seq, verbose=0)
326             preds_models.append(pred.reshape(-1)) # Aseguramos (H,)
327
328         # Bagging simple: promedio entre modelos
329         y_pred_last[i] = np.mean(preds_models, axis=0)
330
331     # ----- VISUALIZACIÓN PARA H=4 -----
332     print("\n== Predicciones futuras (4 semanas) ==")
333
334     # Seleccionamos las 6 empresas con mayor crecimiento previsto en el ú
335     ltimo paso (semana 4)
336     last_values = df_sub.iloc[-1].values
337     diff_abs = y_pred_last[:, -1] - last_values
338     diff_pct = (diff_abs / last_values) * 100
339     top6_idx = np.argsort(diff_pct)[-6:][::-1]
340
341     # ---- TEXTO ----
342     for i in top6_idx:
343         company = df_sub.columns[i]
344         current = last_values[i]
345         preds = y_pred_last[i]
346         print(f"\nEmpresa: {company}")
347         print(f"    Ultimo valor: {current:.2f}")
348         for h, pred in enumerate(preds, start=1):
349             abs_diff = pred - current
350             pct_diff = (abs_diff / current) * 100
351             print(f"    +{h} semana(s): Pred={pred:.2f} | Dif={abs_diff:.2
352                   f} | %+={pct_diff:.2f}%")
353
354     # ---- GRAFICO HISTÓRICO + PREDICCIONES ----

```

```
351 fig, axes = plt.subplots(3, 2, figsize=(14, 12))
352 axes = axes.flatten()
353
354 for j, idx in enumerate(top6_idx):
355     company = df_sub.columns[idx]
356     series = df_sub[company]
357
358     # Histórico
359     axes[j].plot(series.index, series.values, label="Histórico",
360                 color="black")
361
362     # Fechas futuras
363     future_dates = [series.index[-1] + pd.Timedelta(weeks=k) for k in
364                     range(1, H+1)]
365
366     # Unimos último valor real + predicciones
367     full_dates = [series.index[-1]] + future_dates
368     full_preds = [series.values[-1]] + list(y_pred_last[idx])
369
370     # Graficamos predicciones unidas
371     axes[j].plot(full_dates, full_preds, "ro-", label="Predicción (4
372                 semanas)")
373
374     axes[j].set_title(company)
375     axes[j].legend()
376     axes[j].grid(True)
377
378 plt.tight_layout()
379 plt.show()
```

D. Código combinado con H=1 y H=4

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_error, explained_variance_score
7 from sklearn.multioutput import MultiOutputRegressor
8 from sklearn.model_selection import ParameterGrid, TimeSeriesSplit
9 from sklearn.base import clone
10
11 from xgboost import XGBRegressor
12 from lightgbm import LGBMRegressor
13 from tensorflow.keras.models import Sequential
14 from tensorflow.keras.layers import LSTM, GRU, Conv1D, Flatten, Dropout,
Dense
15 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
16 from itertools import product
17
18 import warnings
19 warnings.filterwarnings("ignore")
20
21 SEED = 42
22 np.random.seed(SEED)
23
24 # =====
25 # CREATE SUPERVISED
26 # =====
27 def create_supervised_with_dates(df, W=25, H=1):
28     X, y, dates = [], [], []
29     for col in df.columns:
30         data = df[col].values
31         idx = df.index
32         for i in range(len(data) - W - H + 1):
33             X.append(data[i:i+W])
34             y.append(data[i+W:i+W+H])
35             dates.append(idx[i+W:i+W+H])
36     return np.array(X), np.array(y), np.array(dates)
37
38 # =====
39 # METRICS
40 # =====
41 def compute_metrics(y_true, y_pred):
42     y_true = np.ravel(y_true)
43     y_pred = np.ravel(y_pred)
44     mse = mean_squared_error(y_true, y_pred)
45     rmse = np.sqrt(mse)
46     mae = mean_absolute_error(y_true, y_pred)

```

```

47     r2 = r2_score(y_true, y_pred)
48     mape = np.mean(np.abs((y_true - y_pred)/(y_true+1e-6))) * 100
49     smape = 100*np.mean(2*np.abs(y_pred-y_true)/(np.abs(y_true)+np.abs(
50         y_pred)+1e-6))
51     evs = explained_variance_score(y_true, y_pred)
52     return {"MSE":mse, "RMSE":rmse, "MAE":mae, "R2":r2, "MAPE":mape, "SMAPE":
53         smape, "EVS":evs}
54
55 # =====
56 # DL BUILDERS
57 # =====
58 def build_lstm(input_shape, units=16, dropout=0.3, H=1, **kwargs):
59     model = Sequential([LSTM(units, input_shape=input_shape),
60         Dropout(dropout),
61         Dense(H)])
62     model.compile(optimizer='adam', loss='mse')
63     return model
64
65 def build_gru(input_shape, units=16, dropout=0.3, H=1, **kwargs):
66     model = Sequential([GRU(units, input_shape=input_shape),
67         Dropout(dropout),
68         Dense(H)])
69     model.compile(optimizer='adam', loss='mse')
70     return model
71
72 def build_conv1d(input_shape, units=16, dropout=0.3, kernel_size=3, H=1,
73     **kwargs):
74     model = Sequential([Conv1D(filters=units, kernel_size=kernel_size,
75         activation='relu', input_shape=input_shape),
76         Flatten(),
77         Dropout(dropout),
78         Dense(H)])
79     model.compile(optimizer='adam', loss='mse')
80     return model
81
82 dl_models = {"LSTM": build_lstm, "GRU": build_gru, "Conv1D": build_conv1d
83 }
84
85 # =====
86 # TSCV: ML
87 # =====
88 def tscv_ml(model, X, y, n_splits=5):
89     X_flat = X.reshape((X.shape[0], -1))
90     tscv = TimeSeriesSplit(n_splits=n_splits)
91     metrics_list = []
92     for train_idx, test_idx in tscv.split(X_flat):
93         X_train, X_test = X_flat[train_idx], X_flat[test_idx]
94         y_train, y_test = y[train_idx], y[test_idx]
95         scaler = StandardScaler()
96         X_train_scaled = scaler.fit_transform(X_train)
97         X_test_scaled = scaler.transform(X_test)

```



```

93     model_fold = MultiOutputRegressor(clone(model)) if y.shape[1] > 1
94         else clone(model)
95     model_fold.fit(X_train_scaled, y_train if y_train.shape[1] > 1
96         else np.ravel(y_train))
97     y_pred = model_fold.predict(X_test_scaled)
98     metrics_list.append(compute_metrics(y_test, y_pred))
99     return {k: np.mean([m[k] for m in metrics_list]) for k in
100         metrics_list[0]}
101
102 # =====
103 # TSCV: DL
104 # =====
105 def tscv_dl(model_builder, X, y, n_splits=3, units=16, dropout=0.3,
106     epochs=10, batch_size=32, kernel_size=3):
107     X_seq = X.reshape((X.shape[0], X.shape[1], 1))
108     tscv = TimeSeriesSplit(n_splits=n_splits)
109     metrics_list = []
110     for train_idx, test_idx in tscv.split(X_seq):
111         X_train, X_test = X_seq[train_idx], X_seq[test_idx]
112         y_train, y_test = y[train_idx], y[test_idx]
113         scaler = StandardScaler()
114         X_train_2d = X_train.reshape((X_train.shape[0], -1))
115         X_test_2d = X_test.reshape((X_test.shape[0], -1))
116         X_train_scaled = scaler.fit_transform(X_train_2d).reshape(X_train
117             .shape)
118         X_test_scaled = scaler.transform(X_test_2d).reshape(X_test.shape)
119         model = model_builder(input_shape=(X_train_scaled.shape[1], 1),
120             units=units, dropout=dropout, H=y_train.
121             shape[1], kernel_size=kernel_size)
122         es = EarlyStopping(monitor='val_loss', patience=2,
123             restore_best_weights=True)
124         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
125             =1)
126         model.fit(X_train_scaled, y_train, validation_data=(X_test_scaled
127             , y_test),
128             epochs=epochs, batch_size=batch_size, verbose=0,
129             callbacks=[es, lr])
130         y_pred = model.predict(X_test_scaled, verbose=0)
131         metrics_list.append(compute_metrics(y_test, y_pred))
132     return {k: np.mean([m[k] for m in metrics_list]) for k in
133         metrics_list[0]}
134
135 # =====
136 # RUN EXPERIMENT
137 # =====
138 def run_experiment(path_csv, W=25, H=1, sub_sample_ratio=0.4):
139     np.random.seed(SEED)
140     df_weekly = pd.read_csv(path_csv, index_col=0, parse_dates=True)
141     df_weekly.index = pd.to_datetime(df_weekly.index, utc=True)

```

```

132 sampled_cols = np.random.choice(df_weekly.columns, int(len(df_weekly.
133 columns)*sub_sample_ratio), replace=False)
134 df_sub = df_weekly[sampled_cols]
135
136 X_sub, y_sub, _ = create_supervised_with_dates(df_sub, W=W, H=H)
137 X_sub, y_sub = np.nan_to_num(X_sub), np.nan_to_num(y_sub)
138
139 xgb_params = {"n_estimators": [100, 200], "max_depth": [3, 5], "
140 learning_rate": [0.01, 0.05]}
141 lgb_params = {"n_estimators": [100, 200], "max_depth": [5, 10], "
142 learning_rate": [0.01, 0.05]}
143 dl_params = {'units': [16, 32], 'dropout': [0.2, 0.3]}
144 cld_params = {'units': [16, 32], 'dropout': [0.2, 0.3], 'kernel_size'
145 : [3, 5]}
146
147 ml_models = {"XGB": XGBRegressor, "LGBM": LGBMRegressor}
148 results, best_models = {}, {}
149
150 # ML
151 for name, ModelClass in ml_models.items():
152     best_score, best_model = -np.inf, None
153     param_grid = xgb_params if name=="XGB" else lgb_params
154     for params in ParameterGrid(param_grid):
155         print(f"\nEntrenando {name} con {params}")
156         model = ModelClass(random_state=SEED, verbosity=0, **params)
157         if name=="XGB" else ModelClass(random_state=SEED, verbose=-1,
158 **params)
159         metrics = tscv_ml(model, X_sub, y_sub, n_splits=3)
160         print(f"R2={metrics['R2']:.4f}")
161         if metrics['R2'] > best_score:
162             best_score, best_model = metrics['R2'], model
163     results[name] = best_score
164     best_models[name] = best_model
165
166 # DL
167 for name, builder in dl_models.items():
168     best_score, best_config = -np.inf, None
169     combos = product(cld_params['units'], cld_params['dropout'],
170 cld_params['kernel_size']) if name=="Conv1D" else product(
171 dl_params['units'], dl_params['dropout'])
172     for combo in combos:
173         config = {'units': combo[0], 'dropout': combo[1]} if name!="
174 Conv1D" else {'units': combo[0], 'dropout': combo[1], '
175 kernel_size': combo[2]}
176         print(f"\nEntrenando {name} con {config}")
177         metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3, **config)
178         print(f"R2={metrics['R2']:.4f}")
179         if metrics['R2'] > best_score:
180             best_score, best_config = metrics['R2'], config
181     results[name] = best_score

```

```

172         best_models[name] = best_config
173
174     return results, best_models, df_sub
175
176     # =====
177     # BAGGING + FORECAST
178     # =====
179 def run_bagging_and_forecast(df_sub, W=25, H=1, best_models=None,
180                             top_models=None, n_bags=5):
181     if top_models is None:
182         top_models = ["XGB", "Conv1D"]
183
184     X, y, _ = create_supervised_with_dates(df_sub, W=W, H=H)
185     X, y = np.nan_to_num(X), np.nan_to_num(y)
186     X_flat = X.reshape((X.shape[0], -1))
187
188     tscv = TimeSeriesSplit(n_splits=5)
189     train_idx, test_idx = list(tscv.split(X_flat))[-1]
190     X_train, X_test = X_flat[train_idx], X_flat[test_idx]
191     y_train, y_test = y[train_idx], y[test_idx]
192
193     scaler = StandardScaler()
194     X_train = scaler.fit_transform(X_train)
195     X_test = scaler.transform(X_test)
196
197     # Bagging
198     bagging_preds = np.zeros_like(y_test, dtype=float)
199     for name in top_models:
200         preds_bag = np.zeros_like(y_test, dtype=float)
201         for _ in range(n_bags):
202             if name in ["XGB", "LGBM"]:
203                 base = clone(best_models[name])
204                 if H > 1:
205                     model = MultiOutputRegressor(base)
206                     model.fit(X_train, y_train)
207                     preds = model.predict(X_test)
208                 else:
209                     base.fit(X_train, np.ravel(y_train))
210                     preds = base.predict(X_test).reshape(-1,1)
211             else:
212                 X_train_seq = X_train.reshape((X_train.shape[0], W, 1))
213                 X_test_seq = X_test.reshape((X_test.shape[0], W, 1))
214                 config = best_models[name]
215                 model = dl_models[name](input_shape=(W,1), H=H, **config)
216                 es = EarlyStopping(monitor='val_loss', patience=3,
217                                   restore_best_weights=True)
218                 lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
219                                       patience=2)
220                 model.fit(X_train_seq, y_train, validation_data=(
221                     X_test_seq, y_test),

```

```

218         epochs=20, batch_size=32, verbose=0, callbacks
219         =[es, lr])
220         preds = model.predict(X_test_seq, verbose=0)
221         preds_bag += preds
222         preds_bag /= n_bags
223         bagging_preds += preds_bag
224         bagging_preds /= len(top_models)
225
226     metrics = compute_metrics(y_test, bagging_preds)
227     print(f"Métricas Bagging (H={H}):", {k: round(v,4) for k,v in metrics
228     .items()})
229
230     # Reentrenamiento con todo
231     X_full, y_full, _ = create_supervised_with_dates(df_sub, W=W, H=H)
232     X_full, y_full = np.nan_to_num(X_full), np.nan_to_num(y_full)
233     X_full_flat = X_full.reshape((X_full.shape[0], -1))
234     X_full_scaled = scaler.fit_transform(X_full_flat)
235
236     trained_models = {}
237     for name in top_models:
238         if name in ["XGB", "LGBM"]:
239             base = clone(best_models[name])
240             if H > 1:
241                 model = MultiOutputRegressor(base)
242                 model.fit(X_full_scaled, y_full)
243             else:
244                 base.fit(X_full_scaled, np.ravel(y_full))
245                 model = base
246         else:
247             X_full_seq = X_full_scaled.reshape((X_full_scaled.shape[0], W
248             , 1))
249             config = best_models[name]
250             model = dl_models[name](input_shape=(W,1), H=H, **config)
251             es = EarlyStopping(monitor='val_loss', patience=3,
252             restore_best_weights=True)
253             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
254             patience=2)
255             model.fit(X_full_seq, y_full, epochs=20, batch_size=32,
256             verbose=0, callbacks=[es, lr])
257             trained_models[name] = model
258
259     # Predicción futura
260     n_companies = df_sub.shape[1]
261     y_pred_last = np.zeros((n_companies, H)) if H > 1 else np.zeros(
262     n_companies)
263
264     for i, company in enumerate(df_sub.columns):
265         series = df_sub[company].values
266         last_window = series[-W:].reshape(1, -1)
267         last_window_scaled = scaler.transform(last_window)

```

```

262     preds_models = []
263     for name, model in trained_models.items():
264         if name in ["XGB", "LGBM"]:
265             preds_models.append(model.predict(last_window_scaled))
266         else:
267             last_seq = last_window_scaled.reshape((1, W, 1))
268             preds_models.append(model.predict(last_seq, verbose=0))
269     preds_models = [np.ravel(p) for p in preds_models]
270     preds_models = np.vstack(preds_models)
271     y_pred_last[i] = preds_models.mean(axis=0)
272
273     return y_pred_last, metrics
274
275     # =====
276     # RECURSIVE FORECAST (H=1 -> varios pasos)
277     # =====
278     def recursive_forecast_H1(df_sub, trained_model_H1, scaler, W=25, steps
279     =4):
280         n_companies = df_sub.shape[1]
281         y_pred_recursive = np.zeros((n_companies, steps))
282
283         for i, company in enumerate(df_sub.columns):
284             series = df_sub[company].values
285             window = series[-W:].reshape(1, -1)
286             for s in range(steps):
287                 window_scaled = scaler.transform(window)
288                 if hasattr(trained_model_H1, "predict"): # ML
289                     pred = trained_model_H1.predict(window_scaled)
290                 else: # DL
291                     pred = trained_model_H1.predict(window_scaled.reshape((1,
292                     W, 1)), verbose=0)
293                 pred = float(np.ravel(pred))
294                 y_pred_recursive[i, s] = pred
295                 window = np.roll(window, -1)
296                 window[0, -1] = pred
297             return y_pred_recursive
298
299     # =====
300     # PLOT FORECASTS
301     # =====
302     def plot_forecasts_full(df_sub, y_pred_last_H1, y_pred_last_H4, H1=1, H4
303     =4, title="Predicciones (Full)":
304         last_values = df_sub.iloc[-1].values
305         diff_abs = y_pred_last_H4[:, -1] - last_values
306         top6_idx = np.argsort(diff_abs)[-6:][::-1]
307
308         fig, axes = plt.subplots(3, 2, figsize=(14, 10))
309         axes = axes.flatten()
310         for j, idx in enumerate(top6_idx):
311             company = df_sub.columns[idx]
312             series = df_sub[company]

```

```

310     last_date = series.index[-1]
311     axes[j].plot(series.index, series.values, color="black", label="
Histórico")
312     next_date = last_date + pd.Timedelta(weeks=1)
313     pred_val_H1 = float(y_pred_last_H1[idx])
314     axes[j].plot([last_date, next_date],
315                 [series.values[-1], pred_val_H1],
316                 "bo--", label=f"Pred H={H1}")
317     future_dates = [last_date + pd.Timedelta(weeks=k) for k in range
(1,H4+1)]
318     full_dates = [last_date] + future_dates
319     full_preds = [series.values[-1]] + list(y_pred_last_H4[idx])
320     axes[j].plot(full_dates, full_preds, "ro-", label=f"Pred H={H4}")
321     axes[j].set_title(company)
322     axes[j].legend()
323     axes[j].grid(True)
324     plt.suptitle(title)
325     plt.tight_layout()
326     plt.show()
327
328 def plot_forecasts_zoom(df_sub, y_pred_last_H1, y_pred_last_H4, H1=1, H4
=4, title="Predicciones (Zoom)":
329     last_values = df_sub.iloc[-1].values
330     diff_abs = y_pred_last_H4[:, -1] - last_values
331     top6_idx = np.argsort(diff_abs)[-6:][::-1]
332
333     fig, axes = plt.subplots(3,2, figsize=(14,10))
334     axes = axes.flatten()
335     for j, idx in enumerate(top6_idx):
336         company = df_sub.columns[idx]
337         series = df_sub[company]
338         last_date = series.index[-1]
339
340         # H=1 recursivo
341         future_dates = [last_date + pd.Timedelta(weeks=k) for k in range
(1,H4+1)]
342         full_dates = [last_date] + future_dates
343         full_preds_H1 = [series.values[-1]] + list(y_pred_last_H1[idx])
344         axes[j].plot(full_dates, full_preds_H1, "bo--", label=f"Pred H=1
recursivo")
345
346         # H=4 directo
347         full_preds_H4 = [series.values[-1]] + list(y_pred_last_H4[idx])
348         axes[j].plot(full_dates, full_preds_H4, "ro-", label=f"Pred H=4
directo")
349
350         axes[j].set_title(company)
351         axes[j].legend()
352         axes[j].grid(True)
353         axes[j].tick_params(axis="x", rotation=45)
354

```

```

355     plt.suptitle(title)
356     plt.tight_layout()
357     plt.show()
358
359     # =====
360     # MAIN
361     # =====
362     if __name__=="__main__":
363         path_csv = "stock_weekly_clean.csv"
364
365         # ----- H=1 -----
366         print("\n=== Experimento H=1 ===")
367         res1, models1, df_sub1 = run_experiment(path_csv, H=1)
368         y_pred_last_H1, metrics_H1 = run_bagging_and_forecast(df_sub1, W=25,
369         H=1, best_models=models1)
370
371         # Guardamos scaler y modelo H=1 para el forecast recursivo
372         X_full, y_full, _ = create_supervised_with_dates(df_sub1, W=25, H=1)
373         X_full = np.nan_to_num(X_full)
374         X_full_flat = X_full.reshape((X_full.shape[0], -1))
375         scaler_H1 = StandardScaler().fit(X_full_flat)
376
377         best_model_name = max(res1, key=res1.get)
378         if best_model_name in ["XGB", "LGBM"]:
379             trained_model_H1 = clone(models1[best_model_name])
380             trained_model_H1.fit(scaler_H1.transform(X_full_flat), np.ravel(
381             y_full))
382         else:
383             X_full_seq = scaler_H1.transform(X_full_flat).reshape((X_full.
384             shape[0], 25, 1))
385             config = models1[best_model_name]
386             trained_model_H1 = dl_models[best_model_name](input_shape=(25,1),
387             H=1, **config)
388             es = EarlyStopping(monitor='val_loss', patience=3,
389             restore_best_weights=True)
390             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
391             =2)
392             trained_model_H1.fit(X_full_seq, y_full, epochs=20, batch_size
393             =32, verbose=0, callbacks=[es, lr])
394
395             y_pred_last_H1_recursive = recursive_forecast_H1(df_sub1,
396             trained_model_H1, scaler_H1, W=25, steps=4)
397
398             # ----- H=4 -----
399             print("\n=== Experimento H=4 ===")
400             res4, models4, df_sub4 = run_experiment(path_csv, H=4)
401             y_pred_last_H4, metrics_H4 = run_bagging_and_forecast(df_sub4, W=25,
402             H=4, best_models=models4)
403
404             # ----- Comparaciones -----
405             print("\nComparacion R2 medio por modelo:")

```

```
397     for k in set(res1.keys()) | set(res4.keys()):
398         print(f"{k}: H=1: {res1.get(k, '-')}, H=4: {res4.get(k, '-')}")
399
400     print("\n=== Gráfico comparando H=1 recursivo vs H=4 directo ===")
401     plot_forecasts_zoom(df_sub4, y_pred_last_H1_recursive, y_pred_last_H4
402                         ,
                        H1=1, H4=4, title="Zoom: H=1 recursivo vs H=4
                        directo")
```